

Anno Accademico 2025/2026

Leonardo Donati

Appunti
Informatica Industriale

Teoria e esercizi.



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Indice

1	Introduzione al corso	6
1.1	Mutuazioni e struttura del corso	6
1.1.1	Conoscenza concettuale e tecnologica	6
1.1.2	Capacità applicative	6
1.1.3	Programmazione (parte pratica)	6
1.2	Industry 4.0 e Industry 5.0	7
1.3	Intelligenza artificiale in ambito industriale	7
1.3.1	Manutenzione predittiva	7
1.3.2	Tipi di apprendimento nel Machine Learning	7
1.3.3	Ruolo degli LLM in Industry 5.0	8
1.4	Sistemi di controllo industriale e PLC	8
1.4.1	Caratteristiche di un sistema di controllo industriale	8
1.4.2	Prima generazione: relè e cablaggi complessi	8
1.4.3	PLC: nascita ed evoluzione	9
1.4.4	La programmazione dei PLC prima dell'IEC 61131-3	9
1.4.5	La norma IEC 61131-3: vantaggi	9
1.5	Ingegneria del software nella programmazione dei PLC	10
1.5.1	FSM e modularità	10
1.6	Robotica collaborativa (anteprima)	11
1.7	Analisi del pericolo e sua mitigazione	11
2	Structured Text: strutture di flusso e controllo	12
2.1	Sequenza (esecuzione in ordine)	12
2.2	Selezione: IF / ELSIF / ELSE	12
2.3	Selezione multipla: CASE ... OF	12
2.4	Iterazione: FOR ... DO	13
2.5	Iterazione: WHILE ... DO (con cautela)	13
2.6	Iterazione: REPEAT ... UNTIL	13
2.7	Uscita dai cicli: EXIT (e talvolta CONTINUE)	13
2.8	Blocchi e commenti	13
2.9	Pattern tipico PLC: macchina a stati con CASE	14
2.10	Operatori logici e di confronto	14
2.11	Tipi di dati principali in ST	15
2.11.1	Tipi logici e numerici	15
2.11.2	Tipi temporali (molto usati nei PLC)	15
2.11.3	Tipi a bit e testi	15
2.11.4	Strutture dati	15
3	Programmazione a modalità mutuamente esclusive	16
3.1	Idea generale	16
3.2	Transizioni	16
3.3	Esempio: due modalità con contatore	17
4	Formalismi a stati finiti (prima parte)	18
4.1	Perché usare un formalismo a stati	18
4.1.1	Vantaggi principali	18
4.2	Esempio: data sheet del timer TON vs. diagramma a stati	18
4.2.1	Data sheet del TON	18
4.2.2	Rappresentazione con diagramma a stati	18

4.3	Statecharts	19
4.3.1	Stati XOR (decomposizione gerarchica)	19
4.3.2	Stati AND (regioni ortogonali)	19
4.3.3	Macchine a stati concorrenti	20
4.3.4	Esempio storico: Citizen Quartz Multi-Alarm III	20
4.3.5	Sintassi Statecharts	20
4.4	Il problema dell'interdipendenza tra regioni ortogonali	21
4.4.1	Esempio: sistema biomedico	21
5	Formalismi a stati finiti (seconda parte): Part-Whole Statecharts	22
5.1	Part-Whole Statecharts (PWS)	22
5.1.1	Observer/Controller Pattern	22
5.2	Come funzionano i PWS	22
5.2.1	Componenti: attuatori e sensori	22
5.2.2	Sistema (intero, <i>Whole</i>)	22
5.3	Proposizioni di stato e verifica della sicurezza	23
5.4	Statecharts vs. PW-Statecharts	23
6	Timer TON	24
6.1	Definizione	24
6.2	Comportamento nel tempo	24
6.3	Programma semplice con timer TON	24
7	Simulazione di un componente con transizioni autonome	26
7.1	Note preliminari	26
7.2	Esempio: la Cover	26
7.3	Modellazione	26
8	Stati temporizzati	28
8.1	Introduzione	28
8.2	Simbolismo grafico	28
8.3	Esempio: semaforo	28
8.4	Spia lampeggiante	28
8.4.1	Implementazione (versione base)	29
8.4.2	Versione 2: stop e restart	29
8.5	Controller semaforico temporizzato	30
8.5.1	Direttrici abilitate a tempo	30
8.5.2	Farm Road	30
8.5.3	Farm Road — Extended Time	30
9	Sicurezza e pericolo: nozioni di base	31
9.1	Classificazione dei sistemi in base alla sicurezza	31
9.2	Sicurezza (Safety)	31
9.3	Pericolo (Hazard)	32
9.3.1	Le due dimensioni del pericolo: Rischio e Danno	32
9.3.2	Valutazione quantitativa e qualitativa	32
9.3.3	Matrice dei rischi	32
9.3.4	Classi di pericolo e accettabilità del rischio	32
9.4	Sistemi di base per garantire la sicurezza	33
9.4.1	Interlock (blocco)	33
9.4.2	Guard (sbarramento)	33
9.4.3	Differenze e similitudini	34
9.5	Ruolo dell'oggetto controllato	34
9.5.1	Pericoli non controllabili	34
9.5.2	Pericoli controllabili	34
9.6	Classificazione dei sistemi in base alla gestione dei guasti	34
9.6.1	Sistemi Fail Safe	34
9.6.2	Watchdog (sistema di sorveglianza)	35
9.6.3	Sistemi Fail Operational	35
9.6.4	Sistemi Fail Silent e Fail Explicit	36
9.6.5	Sistemi Bizantini	36

10 Affidabilità, sicurezza e Fault Analysis: FMEA e FTA	39
10.1 Fault, Error e Failure: definizioni e differenze	39
10.1.1 Catena Fault–Error–Failure	39
10.1.2 Esempi nei vari ambiti	40
10.2 Relazione tra Affidabilità e Sicurezza	40
10.2.1 Differenze concettuali	40
10.2.2 Conseguenze	41
10.3 Analisi dei guasti: FMEA e FTA	41
10.3.1 FMEA — Failure Modes and Effects Analysis	41
10.3.2 FTA — Fault Tree Analysis	42
10.3.3 Esempio ed esercizio: laser industriale con interblocco	42
11 Normative sulla sicurezza funzionale nei sistemi di controllo	44
11.1 Obiettivo delle norme	44
11.2 Terminologia	44
11.3 Le due norme: IEC 62061 e ISO 13849-1	44
11.4 Ruolo del controllo nella sicurezza funzionale	45
11.5 Principi generali nella riduzione del rischio	45
11.6 Guasto pericoloso e random fault	45
11.7 Valutazione delle prestazioni delle funzioni di sicurezza	46
11.7.1 Metodologia: PL/SIL richiesto e ottenuto	46
11.7.2 SIL (IEC 62061)	46
11.7.3 PL (ISO 13849-1)	46
11.7.4 Determinazione qualitativa del PL richiesto (PL_r)	46
11.7.5 Determinazione qualitativa del SIL richiesto	47
11.7.6 Corrispondenza tra PL e SIL	47
11.8 Implementazione della funzione di sicurezza	47
11.8.1 MTBF _d : tempo tra due guasti pericolosi	47
11.8.2 DC — Diagnostic Coverage	47
11.8.3 Categorie (architetture)	47
11.8.4 Relazione tra Categorie, DC, MTTF _d e PL	48
11.9 Acronimi	48
12 Programmazione di funzioni di sicurezza in ST: casi di studio	50
12.1 Introduzione	50
12.2 Esempio 1: Funzione di Arresto di Emergenza (E-Stop)	50
12.2.1 Requisiti normativi	50
12.2.2 Struttura I–L–O della funzione	50
12.2.3 Programma ST	51
12.2.4 Analisi del comportamento	51
12.2.5 Tabella di verità con guasti	51
12.2.6 Perché i contatti NC rendono il sistema fail-safe	51
12.2.7 Gli attuatori: principio fail-safe	52
12.2.8 Categoria di sicurezza della funzione E-Stop	53
12.3 Esempio 2: Funzione di Controllo Interblocco di un Riparo	53
12.3.1 Descrizione	53
12.3.2 Sensori e attuatori	53
12.3.3 Programma ST	53
12.3.4 Analisi del comportamento	54
12.3.5 Categoria di sicurezza della funzione di interblocco	54
12.3.6 Integrazione di sensori e attuatori di sicurezza	54
12.4 PLC di sicurezza (Safety PLC)	54
13 Robotica collaborativa	56
13.1 Cobot: definizione e motivazione	56
13.2 Origine dei cobot: problemi ergonomici	56
13.3 Spazio di lavoro collaborativo (ISO 15066)	56
13.4 Forme differenti di co-lavoro	57
13.4.1 Coexistence (spazio separato)	57
13.4.2 Shared Workspace e Sequential Cooperation	57
13.4.3 Simultaneous Co-Work e Parallel Cooperation	57

13.4.4	Collaboration (contatto fisico)	57
13.5	Operazioni collaborative: i quattro metodi	57
13.5.1	Hand Guiding	58
13.5.2	Safety Monitored Stop	58
13.5.3	Speed and Separation Monitoring	58
13.5.4	Power and Force Limiting	58
13.6	Misure protettive in robotica collaborativa	58
13.6.1	Mantenimento della distanza di separazione sufficiente	59
13.6.2	Calcolo della distanza protettiva S_p	59

Capitolo 1

Introduzione al corso

1.1 Mutuazioni e struttura del corso

Il corso di Informatica Industriale (A.A. 2025/26, Prof. Luca Pazzi, DIF UNIMORE) nasce dalla necessità di **armonizzare il curriculum informatico e quello meccanico** (mutuazioni 25/26), con i relativi problemi e opportunità.

Il corso mira a fornire tre tipi di nozioni:

1. **Conoscenza concettuale e tecnologica;**
2. **Capacità applicative;**
3. **Programmazione** (parte pratica, tramite esercitazioni).

1.1.1 Conoscenza concettuale e tecnologica

- Paradigma di **Industry 4.0** (verso 5.0) e **Cyber Physical Systems** per il miglioramento di prodotti e processi;
- **Robotica collaborativa** e **Internet of Things** applicati alla manifattura avanzata;
- Tecnologie informatiche correlate;
- **Ingegneria del software** applicata al ciclo produttivo;
- Nozioni di sicurezza ed efficacia (*safety & liveness*) nei sistemi di produzione;
- Standard internazionali nel settore della sicurezza applicati alla manifattura avanzata.

1.1.2 Capacità applicative

- Progettare le funzioni di un sistema di manifattura avanzata;
- Utilizzare metodologie e standard (**IEC 61508**, **ISO 15066**) per quantificare i rischi e gestire guasti in sistemi distribuiti;
- Verificare la conformità dei progetti agli standard internazionali;
- Integrare l'*heritage* aziendale con le nuove tecnologie e programmare sistemi IoT per la manifattura avanzata.

1.1.3 Programmazione (parte pratica)

- Programmazione di **PLC** attraverso i linguaggi standard **IEC 61131-3**;
- **Modellazione a stati finiti**;
- Implementazione della **modularità** dei sistemi di controllo;
- Distribuzione di autonomia e controllo a differenti livelli di complessità attraverso gli **Statecharts Parte-Intero** (*PW-Statecharts*);
- Progettazione e programmazione «corretta per costruzione» di sistemi che rispettano predicati di sicurezza e di efficacia.

SVOLGIMENTO ED ESAME

Lezioni: lunedì 13–16 e venerdì 10–12. L'esame comprende una **parte scritta** di progettazione e programmazione IEC 61131-3 e un **orale** sulle nozioni apprese durante il corso.

1.2 Industry 4.0 e Industry 5.0

DEFINITION Industry 4.0

Paradigma di manifattura avanzata: tendenza dell'automazione industriale che integra nuove tecnologie produttive per migliorare le condizioni di lavoro e aumentare la produttività e la qualità produttiva degli impianti.

Industry 5.0 è un'evoluzione del paradigma 4.0 con enfasi su:

- **Collaborazione uomo-macchina:** non solo automazione e digitalizzazione, ma interazione più stretta tra operatori umani e sistemi intelligenti;
- **Sostenibilità e resilienza:** robustezza e continuità operativa; ridondanza e modalità degradate (*graceful degradation*).

Le principali caratteristiche della collaborazione uomo-macchina sono:

1. Cobots (robot collaborativi);
2. Intelligenza Artificiale a supporto delle decisioni e della progettazione;
3. Interfacce Uomo-Macchina (HMI) avanzate;
4. Biosensori e sistemi adattivi;
5. Personalizzazione dei processi produttivi.

1.3 Intelligenza artificiale in ambito industriale

1.3.1 Manutenzione predittiva

L'IA sta rivoluzionando la manutenzione industriale, passando da approcci **reattivi** a strategie **proattive** come la **manutenzione predittiva**, che sfrutta algoritmi di Machine Learning e analisi avanzate per prevedere guasti e ottimizzare le operazioni.

DEFINITION Machine Learning

Branca dell'IA che consente ai computer di apprendere dai dati e migliorare le proprie prestazioni in compiti specifici senza essere esplicitamente programmati, tramite algoritmi capaci di identificare pattern nei dati e fare previsioni o decisioni basate su di essi.

1.3.2 Tipi di apprendimento nel Machine Learning

- **Apprendimento supervisionato:** l'algoritmo è addestrato con un dataset etichettato (ogni input associato a un output desiderato); obiettivo: prevedere l'output corretto per nuovi input. Esempi: classificazione email spam/non spam, previsione prezzi immobiliari.
- **Apprendimento non supervisionato:** l'algoritmo analizza dati non etichettati per identificare pattern nascosti o strutture intrinseche. Esempio: *clustering*, segmentazione dei clienti per comportamenti d'acquisto.
- **Apprendimento per rinforzo:** l'algoritmo interagisce con un ambiente dinamico e apprende tramite ricompense e punizioni. Esempio: giochi, dove l'agente impara strategie ottimali per tentativi ed errori.

1.3.3 Ruolo degli LLM in Industry 5.0

I **Large Language Models** (LLM) sono sistemi di IA addestrati su enormi quantità di dati testuali, in grado di comprendere e generare linguaggio naturale. In Industry 5.0 svolgono un ruolo cruciale in:

1. **Interfacce linguistiche naturali;**
2. **Generazione di codice di controllo;**
3. **Analisi avanzata dei dati.**

Per la generazione di codice di controllo per l'automazione, gli LLM possono essere addestrati su linguaggi specifici di settore, generando configurazioni e script complessi da descrizioni in linguaggio naturale: specifica incrementale da parte umana seguita da controllo e verifica di sicurezza (che richiede un paradigma di progettazione e programmazione adatto).

VANTAGGI E SFIDE DEGLI LLM PER I PLC

Vantaggi: aumento della produttività (automazione di compiti ripetitivi), riduzione degli errori (codice conforme alle best practice), aggiornamento continuo, efficienza e accessibilità (anche professionisti con competenze limitate possono sviluppare sistemi di controllo).

Sfide: garantire che il codice generato sia non solo sintatticamente corretto ma anche **funzionalmente sicuro** per l'applicazione specifica; serve un processo iterativo con verifiche semiautomatiche e feedback umano per validare il codice prodotto.

1.4 Sistemi di controllo industriale e PLC

1.4.1 Caratteristiche di un sistema di controllo industriale

Un sistema di controllo industriale non si limita a reagire immediatamente a uno stimolo, ma deve garantire che l'intero processo funzioni in modo stabile e ottimale. Deve:

- **Monitorare costantemente il mondo reale:** raccogliere dati in tempo reale dai sensori, confrontare i valori misurati con i riferimenti, individuare discrepanze;
- **Agire in feedback:** inviare comandi agli attuatori per correggere il processo;
- **Garantire sicurezza e affidabilità:** prevenire malfunzionamenti e situazioni di rischio.

1.4.2 Prima generazione: relè e cablaggi complessi

- **Origini:** negli anni '50-'60 la logica di controllo era implementata con **relè elettromeccanici** (poi elettronici, a transistor), attivati da correnti elettriche;
- **Cablaggio complesso:** la logica (AND, OR, NOT, ecc.) era realizzata con un intricato sistema di cablaggi, un «puzzle elettrico» estremamente complesso nei sistemi grandi;
- **Limitazioni:** manutenzione e modifica difficoltose (bassa flessibilità); errori di cablaggio o guasti a un singolo relè potevano compromettere l'intero sistema.

Dalle logiche a relè alle logiche a transistor l'obiettivo resta lo stesso: realizzare logica discreta per comandare attuatori a partire da sensori (sequenze, interlock, consenso). Le analogie sono:

- *Unità logica:* relè (bobina + contatti NO/NC) ↔ transistor (ingresso + dispositivo di commutazione ON/OFF);
- *Composizione logica:* serie/parallelo ↔ AND/OR; contatto NC ↔ NOT; autoritenuta (latch) ↔ memoria bistabile/flip-flop;
- *Differenza chiave:* il transistor rende la logica più veloce, compatta e affidabile (meno parti meccaniche), ma la struttura funzionale resta la stessa.

DEFINITION Interlock

Vincolo di interblocco (un «consenso») che impedisce l'attivazione di una funzione o azione finché non sono vere alcune condizioni di sicurezza o di corretto funzionamento. È una protezione logica che evita comandi incompatibili o pericolosi.

L'interlock serve, in automazione, a:

- prevenire azioni fisicamente incompatibili (due attuatori che si ostacolano);
- prevenire azioni pericolose (moto con protezioni aperte, presenza operatore, pressione non ok);
- garantire una sequenza corretta (non avviare il ciclo se manca il pezzo, se non si è in home, ecc.).

TIPI DI INTERLOCK

- **Safety interlock** (sicurezza): blocca per proteggere persone/impianto. Esempi: «porta aperta $\Rightarrow$ motori disabilitati»; «operatore in area $\Rightarrow$ robot a velocità ridotta o stop».
- **Process interlock** (logico/di processo): blocca per evitare errori di processo o danni alla macchina. Esempi: «non avviare la pressa se il pezzo non è in posizione»; «non chiudere la pinza senza consenso dal sensore».
- **Mutual interlock** (mutuo interblocco): impedisce due comandi simultanei incompatibili. Esempio: «valvola A e valvola B non possono essere aperte insieme».

1.4.3 PLC: nascita ed evoluzione

I **Controllori Logici Programmabili** (PLC) nascono alla fine degli anni '60 come risposta alle esigenze di automazione industriale (soprattutto industria automobilistica), per sostituire i sistemi a relè e cablaggi complessi, semplificando e rendendo più flessibile il controllo dei processi. Si sono poi evoluti in potenza di calcolo, interfacce di comunicazione e capacità di integrazione, conformandosi allo standard **IEC 61131-3**. Oggi sono fondamentali per affidabilità, flessibilità e capacità di gestire processi complessi in tempo reale.

1.4.4 La programmazione dei PLC prima dell'IEC 61131-3

1. **Logica a relè (ladder diagram)**: diagrammi a contatti che replicavano il cablaggio dei relè, facilitando la transizione ma mantenendo i problemi di fondo: *complessità $\leftrightarrow$ errori $\leftrightarrow$ pericolo*;
2. **Linguaggi proprietari**: sintassi e ambienti specifici per produttore, codice non portabile;
3. **Approccio hardware-centric**: software strettamente legato alla configurazione hardware;
4. **Interoperabilità limitata**: difficoltà nell'integrazione di sistemi multi-PLC;
5. **Strumenti di sviluppo specifici**: poco intuitivi e con funzionalità limitate;
6. **Scarsa riutilizzabilità**: mancanza di astrazioni comuni.

1.4.5 La norma IEC 61131-3: vantaggi

- **Standardizzazione**: unifica i linguaggi tra diversi PLC;
- **Portabilità**: riuso e migrazione del codice tra sistemi di produttori diversi;
- **Flessibilità**: supporta vari paradigmi (LD, FBD, IL, ST, SFC), grafici e testuali;
- **Manutenzione**: migliora debugging e gestione del ciclo di vita;
- **Interoperabilità**: agevola l'integrazione con altri sistemi industriali.

I CINQUE LINGUAGGI IEC 61131-3

Linguaggi grafici:

- **Ladder Diagram (LD)** (KOP in ambiente Siemens): rappresenta graficamente circuiti elettrici con contatti e bobine; intuitivo per chi ha esperienza in elettrotecnica;
- **Function Block Diagram (FBD)**: blocchi funzionali per operazioni logiche e aritmetiche, visione modulare; intuitivo per chi ha esperienza in elettronica;
- **Sequential Function Chart (SFC)**: suddivide il processo in passi e transizioni, ideale per sequenze operative complesse; abbastanza intuitivo per chi ha esperienza in informatica.

Linguaggi testuali:

- **Instruction List (IL)** (AWL in ambiente Siemens): simile ad assembly, mnemonico, a basso livello; poco intuitivo per tutti;
- **Structured Text (ST)**: sintassi simile a Pascal o C; intuitivo per chi ha esperienza informatica, ma il meccanismo operativo è totalmente differente da quello di un PC.

SVANTAGGI DELL'IL

Sintassi di basso livello (difficoltà di lettura e comprensione), manutenzione complessa (soprattutto in team o passaggi di consegne), tracciamento degli errori laborioso a causa della natura lineare e a basso livello del codice.

1.5 Ingegneria del software nella programmazione dei PLC

- I linguaggi a basso livello (IL) sono meno leggibili; quelli di alto livello (ST) migliorano leggibilità e manutenibilità (es. storico: Margaret Hamilton e i listati assembly del software di controllo del modulo Apollo);
- **Scalabilità**: ST offre modularità e organizzazione del codice per progetti grandi, con moduli indipendenti per facilitare manutenzione, riuso ed evoluzione locale senza impattare globalmente l'architettura;
- **Attenzione all'equivoco**: nella programmazione tradizionale (C, Pascal) la modularità si concentra sulle *funzioni del software*; nei PLC la modularità deve concentrarsi sulla rappresentazione del **comportamento (behavior) dei dispositivi**.

INTUITION Dominare la complessità

La progettazione modulare agisce come antidoto alla complessità: isolando le funzionalità in unità indipendenti si limita la propagazione della complessità globale. Ogni modulo, pur complesso internamente, è trattato come «scatola nera» ben definita: si riducono le interdipendenze e si facilitano manutenzione, debugging ed evoluzione.

MODULARITÀ IN SISTEMI REAL-TIME E PLC

Il problema principale è la gestione di comunicazione e sincronizzazione tra moduli: una modularità mal progettata porta a interdipendenze non evidenti. Assicurare scambio affidabile e tempestivo di dati ed eventi richiede meccanismi di coordinamento sofisticati e un modello chiaro di interazione. *La modularità dipende dal modello e dal linguaggio di programmazione.*

Il modello funzionale strutturato (ST) da solo non basta per sistemi real-time: in molti sistemi RT si usano **macchine a stati finiti** per gestire eventi e transizioni assicurando risposta deterministica. La domanda chiave del corso: *come far convivere aspetti RT e modularità?*

1.5.1 FSM e modularità

Il modello a stati finiti è una scelta naturale per applicazioni real-time e PLC industriali: mappa il sistema in stati distinti e transizioni, offrendo una rappresentazione intuitiva dei processi dinamici; in ambito industriale è uno strumento **ontologico** per descrivere la dinamica dei sistemi.

PUNTI DI FORZA E SFIDE DELLE FSM

Punti di forza: rappresentazione naturale della dinamica (transizioni ben definite, es. da stato operativo a stato di emergenza); chiarezza concettuale (regole di transizione esplicite, verifica della correttezza logica agevolata).

Sfide di modularizzazione: scalabilità e complessità (i diagrammi diventano ingestibili crescendo); interconnessione tra moduli (sincronizzazione di stati e transizioni richiede meccanismi ben definiti); gestione della concorrenza (transizioni deterministiche e non conflittuali).

1.6 Robotica collaborativa (anteprima)

A differenza dei robot industriali tradizionali, che operano separati dagli esseri umani per motivi di sicurezza, i **cobot** lavorano a fianco degli operatori e sono progettati per adattarsi in tempo reale alle azioni umane, rendendo la produzione più flessibile ed efficiente (trattati in dettaglio nel Capitolo 13).

1.7 Analisi del pericolo e sua mitigazione

Le particolarità dei moderni sistemi di produzione pongono nuove sfide alla progettazione della sicurezza:

- robot autonomi e collaborativi richiedono di ripensare gli strumenti tradizionali di *safety analysis*;
- **affidabilità non implica sicurezza!**
- occorre conoscere le nuove problematiche introdotte dai nuovi paradigmi produttivi, gli standard attuali e in rilascio, fare esercizi su casi reali e comprendere il ruolo del software nella sicurezza.

Un sistema di produzione complesso combina componenti con diverse strategie di fallimento: componenti singoli *fail silent* o *fail explicit*, dispositivi ridondanti con votazione dei sensori (*fail operational*), gestione *fail operational* e *fail safe* a livello d'impianto (es. centrale nucleare). Queste categorie sono trattate nel Capitolo 9.

BIBLIOGRAFIA DEL CORSO

Kopetz, *Real-Time Systems, Design Principles for Distributed Real-Time Applications*, Kluwer, 1997; Storey, *Safety Critical Computer Systems*, Pearson Prentice Hall, 1996; Leveson, *Engineering a Safer World*, MIT Press.

Capitolo 2

Structured Text: strutture di flusso e controllo

Lo **Structured Text** (ST) è un linguaggio testuale per PLC (standard **IEC 61131-3**) che permette di esprimere algoritmi in modo strutturato: sequenze di istruzioni, selezione tra alternative, ripetizione controllata di blocchi di codice e organizzazione in stati.

CONTESTO PLC: LO SCAN

Nella maggior parte dei PLC il programma gira in **cicli** (*scan*): ad ogni ciclo si leggono gli ingressi, si esegue la logica e si aggiornano le uscite. Per questo motivo, quando possibile, è preferibile scrivere logiche che avanzano «a passi» (ad esempio una macchina a stati) invece di cicli lunghi che potrebbero allungare il tempo di ciclo.

2.1 Sequenza (esecuzione in ordine)

Le istruzioni sono eseguite dall'alto verso il basso:

```
a := 10;
b := a + 5;
MotorOn := (b > 12);
```

2.2 Selezione: IF / ELSIF / ELSE

Sceglie tra rami alternativi sulla base di condizioni booleane:

```
IF Temp > 80.0 THEN
    Alarm := TRUE;
ELSIF Temp > 60.0 THEN
    Warning := TRUE;
ELSE
    Alarm := FALSE;
    Warning := FALSE;
END_IF;
```

Suggerimenti: usare **ELSIF** per evitare annidamenti profondi; rendere le condizioni leggibili (parentesi e nomi chiari) per facilitare la manutenzione.

2.3 Selezione multipla: CASE ... OF

Utile quando le alternative dipendono dal valore di una singola variabile (modalità, comando, stato):

```
CASE Mode OF
    0: (* Stop *)
        Motor := FALSE;
    1: (* Manuale *)
        Motor := StartButton;
```

```

2: (* Automatico *)
   Motor := AutoEnable AND Ready;
ELSE
   (* Valore non previsto *)
   Motor := FALSE;
END_CASE;

```

2.4 Iterazione: FOR ... DO

Ripete un blocco un numero finito di volte (tipico per scorrere array o fare accumuli):

```

Sum := 0;
FOR i := 1 TO 10 DO
   Sum := Sum + A[i];
END_FOR;

```

2.5 Iterazione: WHILE ... DO (con cautela)

Ripete finché la condizione è vera:

```

WHILE (x < 100) DO
   x := x + 1;
END_WHILE;

```

ATTENZIONE

Nei PLC un WHILE può allungare il tempo di ciclo se la condizione resta vera troppo a lungo. Usarlo solo se si è certi che termini rapidamente (o se l'ambiente/standard locale lo consente in modo sicuro).

2.6 Iterazione: REPEAT ... UNTIL

Esegue il corpo almeno una volta, poi verifica la condizione di uscita:

```

REPEAT
   x := x + 1;
UNTIL x >= 10
END_REPEAT;

```

2.7 Uscita dai cicli: EXIT (e talvolta CONTINUE)

EXIT esce dal ciclo più interno; CONTINUE (se supportato) salta alla prossima iterazione:

```

Found := FALSE;
FOR i := 1 TO N DO
   IF A[i] = Target THEN
      Found := TRUE;
      EXIT; (* esce dal FOR *)
   END_IF;
END_FOR;

```

2.8 Blocchi e commenti

I blocchi sono delimitati dalle parole chiave di chiusura: END_IF, END_CASE, END_FOR, END_WHILE, END_REPEAT. Commenti:

- (* ... *) commento multi-linea (standard e molto usato);
- // ... commento a fine riga (spesso supportato; dipende dal vendor).

2.9 Pattern tipico PLC: macchina a stati con CASE

Per sequenze operative e automazioni a passi, una macchina a stati evita cicli lunghi e rende esplicite le transizioni:

```
CASE State OF
  0: (* Idle *)
    Motor := FALSE;
    IF Start THEN
      State := 10;
    END_IF;
  10: (* Avvio *)
    Motor := TRUE;
    IF SpeedOK THEN
      State := 20;
    END_IF;
  20: (* Run *)
    IF Stop THEN
      State := 0;
    END_IF;
END_CASE;
```

MINI-RIEPILOGO: SCELTA DEL COSTRUTTO

- IF/ELSIF/ELSE: soglie, abilitazioni, logica combinatoria con pochi casi;
- CASE: molte alternative basate su una variabile (modalità/stato);
- FOR: array e conteggi finiti;
- WHILE/REPEAT: raramente nei PLC; solo se terminano rapidamente e con motivazione chiara;
- CASE + State: sequenze e automazioni step-by-step (molto consigliato in ambito PLC).

2.10 Operatori logici e di confronto

Le condizioni in IF/WHILE/REPEAT e le espressioni booleane usano operatori logici (su BOOL) e di confronto. Gli stessi operatori logici possono lavorare **bit-a-bit** su tipi a bit (BYTE/WORD/DWORD/LWORD).

Operatori logici principali (su BOOL):

- NOT A: negazione;
- A AND B: vero se entrambi veri;
- A OR B: vero se almeno uno è vero;
- A XOR B: vero se esattamente uno è vero.

Operatori di confronto più comuni:

- = uguaglianza, <> diverso;
- <, >, <=, >= confronti (tipicamente su numeri; spesso anche su TIME/DATE a seconda del sistema).

PRECEDENZA

Regola pratica: NOT ha precedenza più alta, poi AND, poi XOR, poi OR. In caso di dubbio usare sempre le parentesi per rendere esplicita l'intenzione.

Esempi:

```
(* Interlock logico: abilita uscita solo se non c'è guasto
e la pressione è sufficiente *)
Out := Enable AND NOT Fault AND (Pressure > 2.0);
```

```
(* Test di appartenenza a intervallo *)
InRange := (x >= 10) AND (x <= 20);

(* De Morgan (utile per semplificare/trasformare condizioni) *)
Safe := NOT (A OR B);    (* equivalente a (NOT A) AND (NOT B) *)
```

2.11 Tipi di dati principali in ST

ST usa i tipi standard IEC 61131-3 (nomi e disponibilità possono variare leggermente tra ambienti, ma le famiglie principali sono comuni).

2.11.1 Tipi logici e numerici

- **BOOL**: vero/falso (TRUE/FALSE);
- **Interi**: SINT, INT, DINT, LINT e varianti senza segno (USINT, UINT, UDINT, ULINT);
- **Reali**: REAL (32 bit), LREAL (64 bit).

2.11.2 Tipi temporali (molto usati nei PLC)

- **TIME**: durata (es. T#200ms, T#2s, T#1m);
- **DATE**, **TIME_OF_DAY** (TOD), **DATE_AND_TIME** (DT): data/ora (se supportati e configurati).

2.11.3 Tipi a bit e testi

- **BYTE**, **WORD**, **DWORD**, **LWORD**: bit string; AND/OR/XOR/NOT agiscono bit-a-bit;
- **STRING[n]**, **WSTRING[n]**: stringhe (lunghezza massima *n*).

2.11.4 Strutture dati

- **ARRAY[low..high] OF T**: vettori/matrici (es. misure campionate);
- **STRUCT**: record/strutture (se supportato; utile per raggruppare campi).

Esempio di dichiarazioni (schema tipico):

```
VAR
    Alarm      : BOOL;
    Counter    : DINT;
    Temp       : REAL;
    tDelay     : TIME := T#2s;
    Mode       : INT;
    Name       : STRING[20];
    A          : ARRAY[1..10] OF REAL;
END_VAR
```

CONVERSIONI DI TIPO

Per conversioni tra tipi (ad esempio INT → REAL) si usano funzioni standard come INT_TO_REAL, REAL_TO_INT, ecc. Il nome esatto può dipendere dall'ambiente, ma il concetto è sempre lo stesso: rendere **esplicita** la conversione quando cambia il tipo.

Capitolo 3

Programmazione a modalità mutuamente esclusive

ORIGINE

Appunti dell'esercitazione del 9 marzo 2026. Nota: il programma svolto in aula usa nomi differenti da quelli usati nelle slide.

3.1 Idea generale

Nella programmazione a **task** è utile individuare **porzioni di codice che vengono eseguite in maniera mutuamente esclusiva**. Si usano **variabili booleane** per creare:

- porzioni mutuamente esclusive nel codice (**stati**);
- segnali e condizioni che fanno saltare da una porzione all'altra (**eventi**).

Si usa il costrutto **IF THEN** associato a variabili booleane per creare tali porzioni. Una porzione contiene:

- una o più **azioni**;
- una o più **transizioni**.

Una porzione di codice modella una **modalità**, o meglio uno **stato**.

3.2 Transizioni

DEFINITION Transizione

Una transizione:

- è contenuta in una porzione di codice;
- porta in un'altra porzione di codice;
- è guidata da un **evento**, modellato attraverso un segnale booleano;
- è racchiusa in un **IF THEN**;
- deve essere eseguita **una sola volta**: ciò si ottiene rendendo falso il segnale che modella l'evento alla conclusione dell'**IF** («**consumare l'evento**»).

Una transizione può avvenire anche in «reazione» a una **condizione** che diventa vera (senza eventi): ad esempio si salta alla modalità «indietro» se l'indice **i** ha raggiunto un certo limite.

3.3 Esempio: due modalità con contatore

Programma con due modalità in cui un contatore viene incrementato o decrementato. Si passa da una modalità all'altra se (in alternativa):

- il contatore ha superato un limite negativo o positivo ($-1000, 1000$);
- un evento **step** viene inviato attraverso un button (pulsante) nella parte grafica.

Dichiarazione delle variabili:

```
PROGRAM PLC_PRG
VAR
    i : INT := 0;
    //-- Modalita' oppure STATI
    avanti : BOOL := TRUE;
    indietro : BOOL := FALSE;
    //-- Evento
    step : BOOL := FALSE;
END_VAR
```

Le variabili booleane **avanti** e **indietro** rappresentano gli **stati** (modalità) ed è garantito che siano sempre mutuamente esclusivi (uno **TRUE** e l'altro **FALSE**); **step** è l'**evento** esterno che forza il cambio di modalità indipendentemente dal valore del contatore.

INTUITION Perché questo pattern è importante

Questo schema (stati = porzioni di codice mutuamente esclusive, transizioni guidate da eventi o condizioni) è la traduzione diretta in ST di una **macchina a stati**: è il ponte tra il formalismo a stati finiti (Capitoli 4 e 5) e la programmazione concreta dei PLC, e sarà riutilizzato in tutti i casi di studio successivi (timer, stati temporizzati, funzioni di sicurezza).

Capitolo 4

Formalismi a stati finiti (prima parte)

4.1 Perché usare un formalismo a stati

Un **dispositivo reattivo** evolve nel tempo, passando fra diverse modalità di funzionamento in risposta a eventi, comandi e condizioni dell'ambiente.

Un formalismo a stati permette di rappresentare il comportamento come:

- un **insieme finito di stati significativi** (ad es. spento, pronto, in lavoro, errore);
- **transizioni** fra stati, attivate da eventi e condizioni osservate (**guardie**).

4.1.1 Vantaggi principali

- Rende il comportamento più chiaro di una descrizione solo testuale;
- separa le modalità operative (stati) dalle singole azioni elementari;
- evidenzia *quando* e *perché* il sistema cambia stato;
- facilita l'analisi di coerenza, completezza e sicurezza.

Il modello a stati non descrive soltanto *che cosa* il dispositivo fa, ma soprattutto *in quali condizioni* lo fa e *come* passa da una situazione all'altra.

4.2 Esempio: data sheet del timer TON vs. diagramma a stati

4.2.1 Data sheet del TON

La data sheet del timer TON descrive la relazione fra le variabili (IN, ET, PT, Q):

- descrive correttamente il TON in termini di variabili e condizioni;
- richiede però di ricostruire mentalmente la sequenza del comportamento;
- è meno immediata per capire «in che fase si trova» il timer.

4.2.2 Rappresentazione con diagramma a stati

Il diagramma a stati mostra subito le tre situazioni essenziali (**Reset**, **TimeIn** ovvero conteggio, **TimeOut**) e rende esplicite le condizioni di passaggio:

- **IN** := TRUE (ingresso in conteggio);
- **IN** := FALSE (ritorno a reset);
- **ET** > **PT** (timeout).

Facilita comprensione, spiegazione e verifica del comportamento.

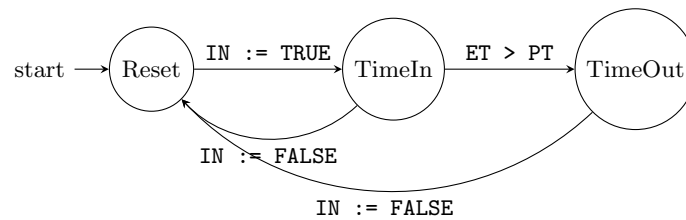


Figura 4.1: Comportamento del timer TON come diagramma a stati: Reset ($IN = FALSE$, $Q = FALSE$), TimeIn (conteggio, $Q = FALSE$, $ET \leq PT$), TimeOut ($Q = TRUE$, $ET > PT$).

4.3 Statecharts

DEFINITION Statechart

Estensione dei diagrammi a stati finiti, progettata per modellare sistemi complessi e reattivi. Introdotto da **David Harel nel 1987** per affrontare la complessità dei sistemi reattivi (come quelli avionici). Fornisce una rappresentazione visiva e modulare del comportamento di sistemi complessi.

Caratteristiche principali:

- **Gerarchia:** decomposizione dei sistemi in sottostati, per gestire la complessità;
- **Concorrenza:** stati paralleli (regioni ortogonali) per modellare comportamenti simultanei;
- **Transizioni inter-livello:** transizioni tra stati a diversi livelli della gerarchia.

Applicazioni: sistemi embedded e in tempo reale, protocolli di comunicazione, interfacce uomo-macchina. Vantaggi: riduzione della complessità attraverso la modularità, facilità di comprensione e manutenzione, supporto per simulazione e verifica formale.

4.3.1 Stati XOR (decomposizione gerarchica)

DEFINITION Stato XOR

Uno stato XOR (o stato composto) rappresenta una gerarchia in cui, entrando nello stato superiore, il sistema si trova in **uno solo** dei suoi sottostati alla volta.

Caratteristiche:

- modellano comportamenti mutuamente esclusivi;
- riducono la complessità evitando la duplicazione di transizioni comuni;
- esempio: in un lettore musicale lo stato «Riproduzione» può avere sottostati «In pausa», «In riproduzione», «Arrestato»; il sistema è in uno solo di questi alla volta.

4.3.2 Stati AND (regioni ortogonali)

DEFINITION Stato AND

Uno stato AND (o stato con regioni ortogonali) rappresenta una decomposizione parallela in cui, entrando nello stato superiore, il sistema si trova **simultaneamente in tutti** i suoi sottostati.

Caratteristiche:

- modellano comportamenti concorrenti o indipendenti;
- ogni regione ortogonale può evolversi indipendentemente dalle altre;
- evitano la combinazione esplosiva di stati (niente moltiplicazione di stati per ogni combinazione possibile).

CONFRONTO XOR VS. AND

Caratteristica	Stato XOR	Stato AND
Attività simultanee	No	Sì
Complessità	Riduce la duplicazione	Gestisce la concorrenza
Indipendenza	Stati mutuamente esclusivi	Stati indipendenti e paralleli
Utilizzo tipico	Macchine a stati	Comportamenti concorrenti

4.3.3 Macchine a stati concorrenti

Usando appropriatamente i due meccanismi di decomposizione si costruiscono **macchine a stati concorrenti**: ciascuna macchina è ospitata in una sezione AND di uno stato decomposto. Dato che una macchina si trova in un unico stato alla volta, ogni sezione AND viene decomposta in una serie di stati XOR che costituiscono gli stati della macchina in essa contenuta.

4.3.4 Esempio storico: Citizen Quartz Multi-Alarm III

L'esempio classico (dall'articolo di Harel *Statecharts: A Visual Formalism for Complex Systems*) è il modello comportamentale dell'orologio Citizen Quartz Multi-Alarm III, ottimo esempio di decomposizione XOR e AND combinata.

Lo statechart mostra lo stato principale **main** (e lo stato secondario **dead**), all'interno del quale si trovano sei regioni ortogonali (AND): quando il sistema è in **main**, tutte le regioni sono attive contemporaneamente ed ognuna evolve indipendentemente:

1. **main**
2. **alarm1-status**
3. **alarm2-status**
4. **chime-status**
5. **light**
6. **power**

Decomposizione AND (mutuamente parallela): le regioni AND sono separate da linee tratteggiate verticali all'interno di **main**. Ad esempio **alarm1-status** e **alarm2-status** funzionano in parallelo: ciascuna può essere **enabled** o **disabled** indipendentemente dall'altra. **chime-status** è più complessa: ha uno stato **enabled** al cui interno si trova una decomposizione OR (**quiet** e **beep**).

Decomposizione XOR (mutuamente esclusiva): alcune regioni usano la decomposizione XOR per i propri sottostati: in **alarm1-status** e **alarm2-status** il sistema è in uno e uno solo tra **enabled** e **disabled**; in **chime-status** si entra in un sottostato che può essere **quiet** oppure **beep** (2 secondi, a ogni ora intera).

Eventi e transizioni: il diagramma usa eventi esterni (**batt.inserted**, **batt.removed**, **batt.weakens**, **in.alarm1.on**, ecc.), azioni (es. **/clh(main*)** indica il ritorno allo stato principale con azione **clh**) e condizioni (es. **T is whole hour**).

4.3.5 Sintassi Statecharts

- **Eventi**: fanno avvenire le transizioni. Esempio: se la macchina a stati nella sezione parallela **valve** si trova in **Open** ed è presente l'evento **close**, passa a **Closed**;
- **Guardie**: condizioni booleane sullo stato di altre macchine, associate alle transizioni; costituiscono **condizione necessaria** affinché la transizione avvenga. Esempio: se **valve** è in **Closed** ed è presente l'evento **open**, passa a **Open** se e solo se la sezione **pressure** (abbreviata **p**) è in **Low**, scritto **[p=Low]**;
- **Eventi broadcast**: gli eventi preceduti dal simbolo **</>** vengono inviati a tutte le altre sezioni parallele, eventualmente avviandovi una transizione. Esempio: se **pump** è in **Stop** ed è presente **start**, passa in marcia inviando l'evento **open** alle altre sezioni.

4.4 Il problema dell'interdipendenza tra regioni ortogonali

ASPETTATIVA TEORICA VS. REALTÀ

Nella **decomposizione AND** le regioni dovrebbero evolversi in parallelo, ognuna gestendo il proprio sottoinsieme di stati, concettualmente indipendenti.

Nel **modello del Citizen Quartz** (e in molti sistemi reali) le regioni condividono eventi (`batt.removed`, `in.alarm1.on`, `T is whole hour`) e le transizioni in una regione possono dipendere dallo stato o dal comportamento di un'altra.

Conseguenze dell'interdipendenza:

- **Perdita di modularità:** le regioni non possono essere progettate in isolamento; modifiche a una regione possono introdurre effetti collaterali sulle altre;
- **Aumento del coupling semantico:** anche se il modello è strutturato «in parallelo», la semantica diventa interdipendente e intrecciata; si crea una dipendenza logica tra regioni che rompe l'astrazione parallela.

4.4.1 Esempio: sistema biomedico

Un sistema biomedico composto da tre componenti, ciascuna macchina a stati derivante da decomposizione OR ospitata in tre differenti decomposizioni AND:

- **p:** la pompa di infusione;
- **v:** la valvola antireflusso;
- **ps:** il sensore di pressione.

INTUITION «La complessità che esce dalla porta rientra dalla finestra»

Nonostante il numero minore di stati, le diverse sezioni parallele si devono sincronizzare attraverso relazioni causali mutue (eventi e guardie) di non facile comprensibilità, che limitano la riusabilità delle macchine a stati. Ad esempio la pompa non è una pompa a sé stante, ma una pompa collegata e dipendente da una valvola e da un sensore di pressione. Le sincronizzazioni mutue (freccie grigie nei diagrammi) sono ciò che rende tre parti autonome un'unica macchina.

Questa criticità motiva la seconda parte: i **Part-Whole Statecharts** (Capitolo 5), che separano esplicitamente i comportamenti dei componenti da quello del sistema.

Capitolo 5

Formalismi a stati finiti (seconda parte): Part-Whole Statecharts

5.1 Part-Whole Statecharts (PWS)

DEFINITION Part-Whole Statecharts

Formalismo che **separa il comportamento dei componenti da quello del sistema complessivo**, attraverso due sezioni distinte e parallele:

1. **Assembly** (comportamenti locali dei componenti);
2. **System section** (*Whole*, comportamento globale).

5.1.1 Observer/Controller Pattern

Il pattern di riferimento dei PWS risponde alla domanda: «chi può conoscere/controllare gli altri componenti?»

- **Componenti:** no, neppure il Sistema (Whole) è conosciuto dai componenti. Ogni componente è **comportamentalmente isolato**;
- **Sistema:** sì, conosce e controlla i componenti.

5.2 Come funzionano i PWS

Ogni componente è modellato come un'**automa a stati riusabile e testabile in isolamento**. Il sistema (*Whole*) coordina i componenti tramite:

1. **Eventi diretti** (non broadcast);
2. **Guardie** sulle transizioni.

5.2.1 Componenti: attuatori e sensori

- **Attuatori:** caratterizzati da eventi **trigger** (come **start** e **stop**) che fanno scattare le rispettive transizioni;
- **Sensori:** possono avere **transizioni che scattano da sole**, senza alcun trigger che le faccia scattare (**transizioni autonome**, notate con notazione grafica dedicata).

5.2.2 Sistema (intero, *Whole*)

- È caratterizzato da eventi trigger (come **stop** e **on**) che fanno scattare le rispettive transizioni;
- può avere **guardie** alle transizioni, che costituiscono condizione necessaria affinché la transizione scatti;
- può inviare **eventi diretti** (azioni) alle componenti attraverso **liste di azioni**: ad esempio `< p.stop, v.close >`.

5.3 Proposizioni di stato e verifica della sicurezza

Attraverso **proposizioni di stato** si possono:

- esplicitare **invarianti** (es. «se pressione è alta, la pompa è spenta e la valvola è chiusa»);
- derivare **safety axioms** controllando solo gli stati del sistema (non combinazioni esplosive di stati locali);
- evitare il **model checking pesante**, usando logica proposizionale per garantire sicurezza.

5.4 Statecharts vs. PW-Statecharts

CONFRONTO

Problema Statecharts	Soluzione PW-Statecharts
Perdita di modularità	Isolando completamente i componenti
Complessità della sincronizzazione	Centralizzando logica e coordinamento
Difficoltà di test e certificazione	Permettendo test modulari e semantica globale chiara
Verifica della sicurezza	Tramite proposizioni e transizioni sicure

INTUITION In sintesi

Dove gli Statecharts classici di Harel fallivano (regioni ortogonali interdipendenti, Capitolo 4), i PWS ribaltano l'architettura: i componenti sono automi completamente isolati e riusabili, e tutte le interazioni passano dal sistema (Whole), che osserva e comanda. La sicurezza si verifica a livello di sistema con proposizioni di stato, senza esplorare il prodotto cartesiano degli stati locali.

Capitolo 6

Timer TON

6.1 Definizione

DEFINITION TON — Timer On Delay

Function Block che implementa un ritardo all'accensione (*turn-on delay*). La firma è TON(IN, PT, Q, ET):

- **Input:**

- IN (tipo BOOL): il segnale di «start»; quando viene posto a TRUE il timer inizia a contare;
- PT (tipo TIME): il «Prescribed Time», il tempo prescritto prima che venga emesso il segnale di timeout;

- **Output:**

- Q (tipo BOOL): il segnale di timeout;
- ET (tipo TIME, «Elapsed time»): il tempo trascorso da quando il timer è partito.

6.2 Comportamento nel tempo

- **Inizialmente:** IN = FALSE, Q = FALSE, ET = 0;
- **Appena IN diventa TRUE:** il tempo inizia a essere contato in millisecondi in ET, finché il suo valore non raggiunge PT; poi ET resta costante;
- **Q diventa TRUE** quando IN è TRUE e ET è uguale a PT.

Casi di disattivazione:

- se IN viene posto a FALSE **dopo** la scadenza di PT: Q torna FALSE e ET viene azzerato;
- se IN viene posto a FALSE **prima** della scadenza di PT: Q resta FALSE (non è mai scattato) e ET viene azzerato (il conteggio riparte da capo al prossimo fronte di IN).

6.3 Programma semplice con timer TON

Esempio d'uso tipico (con blocchi numerati come nelle slide):

- il **blocco (1...3)** viene eseguito finché PT non è trascorso e Q resta FALSE; poi Q diventa TRUE;
- il **blocco (4...7)** viene eseguito **una sola volta**...
- ... poiché porre IN a FALSE forza Q a FALSE (autodisattivazione: il ramo che scatta con Q = TRUE spegne il timer).

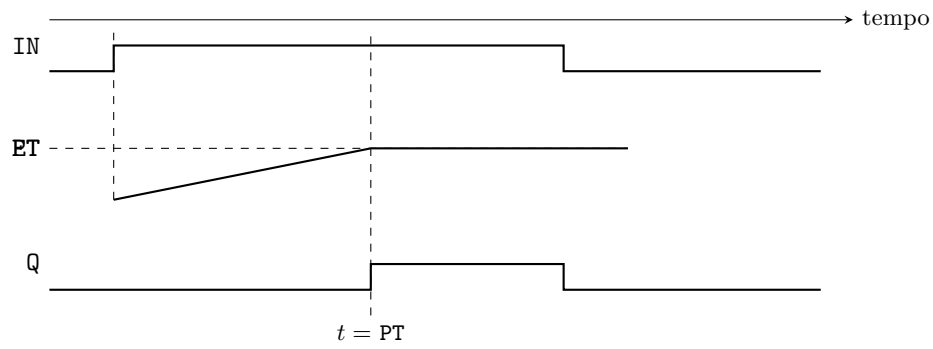


Figura 6.1: Timing diagram del TON: ET cresce linearmente finché raggiunge PT, allora Q sale; quando IN torna FALSE, Q scende e ET si azzerà.

INTUITION Pattern d'uso

Il pattern fondamentale è: *chiamare* il timer ad ogni ciclo con `timer(IN := ..., PT := ...)`, *testare* il timeout con `IF timer.Q THEN ...`, e *resettare* il timer con `timer(IN := FALSE)` all'interno del ramo di timeout. Questo pattern è la base per gli **stati temporizzati** (Capitolo 8).

Capitolo 7

Simulazione di un componente con transizioni autonome

7.1 Note preliminari

- Un controller **PWs** (*Part-Whole system*, Capitolo 5) interagisce con un certo numero di **interfacce di componenti** raccolte in un **assembly**;
- **Interfaccia**: descrive *solo* i cambiamenti di stato del componente, siano essi comandati attraverso **trigger** oppure attraverso **transizioni autonome**;
- Le transizioni autonome riflettono sia cambiamenti nell'**ambiente**, sia cambiamenti che derivano dall'**implementazione**.

7.2 Esempio: la Cover

In generale un dispositivo può **nascere in più di uno stato**. La *cover* (coperchio) nasce per esempio sia nello stato **Up** sia nello stato **Down**: ciò deriva dal fatto che, nel momento in cui il controller a cui è collegata comincia a funzionare, la cover può essere già aperta o chiusa.

La cover può muoversi dallo stato **Up** allo stato **Down** e viceversa **in ogni momento**: ciò deriva dal fatto che la cover può essere aperta o chiusa manualmente (interazione con l'ambiente). È in realtà un **sensore**!

SEMPLIFICAZIONE DIDATTICA

Per semplificare la simulazione a fini didattici verranno usati solo dispositivi con un **solo stato iniziale**. In questo caso si immagina che la Cover sia inizialmente chiusa (**Down**), ad esempio tramite una molla che la tiene abbassata.

7.3 Modellazione

Si dichiarano variabili di input usate come **trigger ad uso esclusivo della simulazione**, usando nomi difficilmente confondibili con quelli reali del device:

```
// ----- FB Cover -----  
FUNCTION_BLOCK Cover  
VAR_INPUT  
    sim_event_Up_Down : BOOL := FALSE; (* simulation event Up->Down *)  
    sim_event_Down_Up : BOOL := FALSE; (* simulation event Down->Up *)  
END_VAR  
VAR_OUTPUT  
    stato : Stati_Cover := Stati_Cover.Init;  
END_VAR
```

Implementazione come macchina a stati con **CASE OF** (transizioni guidate dagli eventi di simulazione, con «consumo» dell'evento):

```

CASE stato OF
  Stati_Cover.Init:
    stato := Stati_Cover.Down;

  Stati_Cover.Up:
    IF sim_event_Up_Down THEN
      stato := Stati_Cover.Down;
      sim_event_Up_Down := FALSE;    (* consumo evento *)
    END_IF;

  Stati_Cover.Down:
    IF sim_event_Down_Up THEN
      stato := Stati_Cover.Up;
      sim_event_Down_Up := FALSE;    (* consumo evento *)
    END_IF;
END_CASE

```

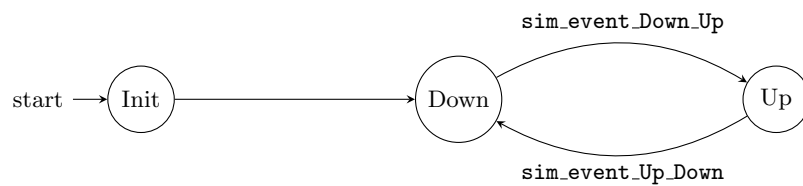


Figura 7.1: Automa del Function Block **Cover**: stato iniziale unico **Init** (semplificazione: molla che tiene la cover abbassata), poi transizioni comandate dagli eventi di simulazione.

INTUITION Punti chiave

- Gli eventi di simulazione sono variabili *distinte* da quelle del dispositivo reale (prefisso **sim_**), così il codice di simulazione non si mescola con la logica reale;
- dopo ogni transizione l'evento viene «consumato» (riportato a **FALSE**), secondo il pattern visto nel Capitolo 3;
- nel sistema reale le transizioni della cover sarebbero **autonome** (scattano da sole, senza trigger del controller): nella simulazione vengono emulate con trigger dedicati.

Capitolo 8

Stati temporizzati

8.1 Introduzione

Il timer TON (Capitolo 6) viene utilizzato attraverso il nuovo concetto di **stato temporizzato**:

- uno stato temporizzato è tale che il controllo rimane in esso per **al più un tempo fissato**;
- scaduto questo tempo, una **transizione autonoma** (detta di **timeout**) porta il controllo verso un altro stato;
- altre transizioni possono portare il controllo fuori dallo stato **prima** del timeout.

8.2 Simbolismo grafico

1. Lo stato temporizzato presenta un **riquadro** che indica il tempo massimo T in cui il controllo rimarrà in esso;
2. la transizione di **timeout** presenta una T e un pallino, indicandone la natura temporale e autonoma.

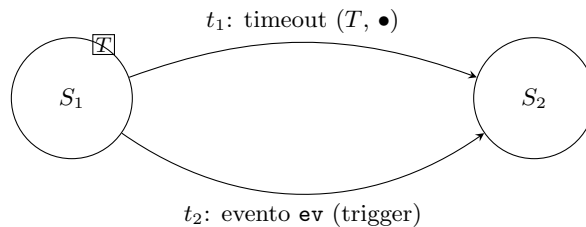


Figura 8.1: Notazione dello stato temporizzato: se l'evento **ev** che agisce da trigger giunge prima dello scadere del tempo T , verrà intrapresa la transizione t_2 ; altrimenti, allo scadere di T , scatterà la transizione autonoma di timeout t_1 .

8.3 Esempio: semaforo

Un primo esempio di stato temporizzato è la realizzazione del semaforo stradale: la transizione da giallo (Y) a rosso (R) è sempre autonoma, ma ora *sappiamo a che tempo avviene* ($T = 5\text{ s}$):

```
Stati_Sem.Y:  
  timer(IN := TRUE, PT := T#5S );  
  IF timer.Q THEN  
    stato := Stati_Sem.R;  
    timer(IN := FALSE);  
  END_IF
```

8.4 Spia lampeggiante

Una spia lampeggiante passa dallo stato acceso (**On**) allo stato spento (**Off**), ciascuno dei due stati ha una durata predefinita: il comportamento globale è dato da **due stati temporizzati collegati tra loro**.

8.4.1 Implementazione (versione base)

```

FUNCTION_BLOCK Light
VAR_OUTPUT
    stato : Light_St := Light_St.Init;
END_VAR
VAR
    timer : TON;
END_VAR

CASE stato OF
    Light_St.Init:
        stato := Light_St.On;

    Light_St.Off:
        timer(IN:=TRUE, PT := T#1S);
        IF timer.Q THEN
            stato := Light_St.On;
            timer(IN := FALSE );
        END_IF

    Light_St.On:
        timer(IN:=TRUE, PT := T#2S);
        IF timer.Q THEN
            stato := Light_St.Off;
            timer(IN := FALSE );
        END_IF
END_CASE

```

REGOLE IMPLEMENTATIVE DELLO STATO TEMPORIZZATO

- Uno stato temporizzato **richiama il timer all'inizio del proprio codice** (ad ogni ciclo di scan);
- la transizione di timeout è attivata dal segnale logico Q del timer: `IF timer.Q THEN ...`;
- **ogni transizione in uscita** (compresa quella di timeout) **termina con il reset del timer**: `timer(IN := FALSE)`.

8.4.2 Versione 2: stop e restart

In questa seconda versione la spia lampeggiante può essere **fermata in ogni momento** con l'evento **stop** e fatta **ripartire** con l'evento **restart**. Ogni stato temporizzato contiene, oltre alla logica di timeout, un IF che verifica l'evento di stop (che consuma l'evento e resetta il timer):

```

FUNCTION_BLOCK Light
VAR_INPUT
    stop_ev, restart_ev : BOOL := FALSE;
END_VAR
VAR_OUTPUT
    stato : Light_St := Light_St.Init;
END_VAR
VAR
    timer : TON;
END_VAR

CASE stato OF
    Light_St.Init:
        stato := Light_St.On;

    Light_St.Off:
        timer(IN:=TRUE, PT := T#2S);
        IF timer.Q THEN
            stato := Light_St.On;
            timer(IN := FALSE );

```

```

END_IF
IF stop_ev THEN
    stato := Light_St.Stop;
    stop_ev := FALSE;
    timer(IN := FALSE);
END_IF

Light_St.On:
    timer(IN:=TRUE, PT := T#1S);
    IF timer.Q THEN
        stato := Light_St.Off;
        timer(IN := FALSE );
    END_IF
    IF stop_ev THEN
        stato := Light_St.Stop;
        stop_ev := FALSE;
        timer(IN := FALSE);
    END_IF

Light_St.Stop:
    IF restart_ev THEN
        stato := Light_St.On;
        restart_ev := FALSE;
    END_IF
END_CASE

```

8.5 Controller semaforico temporizzato

8.5.1 Direttrici abilitate a tempo

Nel controller semaforico visto in precedenza le due direttrici venivano attivate da due eventi trigger. In questa variante le due direttrici **ns** (nord-sud) ed **ew** (est-ovest) sono abilitate **esattamente per un tempo stabilito** (stati temporizzati).

8.5.2 Farm Road

DEFINITION Farm Road

Strada secondaria che si immette su una strada principale (*main road*). La farm road rimane abilitata **a richiesta** (evento **car**) esattamente per un tempo $T = 30$ s.

8.5.3 Farm Road — Extended Time

Variante: la farm road rimane abilitata a richiesta (evento **car**) esattamente per un tempo T ; **mentre** la farm road è abilitata, un secondo evento **car** può **estendere** il tempo di abilitazione di ulteriori 30 secondi **ma solo una volta**.

INTUITION Perché gli stati temporizzati

Gli stati temporizzati rendono *esplicito nel modello* il tempo di permanenza in uno stato, integrando il timer TON nel formalismo a stati: il timeout è una transizione autonoma (scatta da sola, senza trigger esterno), mentre le altre transizioni possono anticiparlo. In ST si implementano con il pattern: chiamata del timer in testa allo stato, test di **timer.Q** per il timeout, reset del timer su ogni uscita, e gestione degli eventi con consumo del segnale.

Capitolo 9

Sicurezza e pericolo: nozioni di base

La sicurezza (*safety*) e i pericoli (*hazards*) associati ai sistemi ingegneristici controllati da computer sono aspetti fondamentali da comprendere per poter progettare sistemi in grado di garantirli.

9.1 Classificazione dei sistemi in base alla sicurezza

DEFINITION Sistemi safety-critical

Sistemi progettati per prevenire incidenti che possono mettere a rischio vite umane o causare danni ambientali significativi. Esempi: sistema di controllo di un reattore nucleare, sistema frenante di un aereo o di un'automobile. In questi casi eventuali guasti devono essere gestiti in modo da portare l'oggetto controllato in uno stato sicuro (**fail-safe**) o, al minimo, segnalare immediatamente il problema (**fail-explicit**).

- **Mission-critical**: sistemi essenziali per il completamento di una missione o di un'operazione specifica;
- **Business-critical**: sistemi il cui fallimento comporta gravi ripercussioni economiche o di reputazione per un'organizzazione.

9.2 Sicurezza (Safety)

DEFINITION Sicurezza

La sicurezza di un sistema è la proprietà di **non recare danno alla vita umana o all'ambiente** durante il suo funzionamento. Garantire la sicurezza è un'attività prevalentemente umana, spesso considerata più un'arte che una scienza nelle fasi iniziali di progettazione.

Per assicurare la sicurezza occorre un approccio integrato che tenga conto di diversi fattori del sistema:

- il **software** (che controlla la logica di funzionamento);
- l'**hardware** (sensori, attuatori, cablaggi, connettori, alimentazione, ecc.);
- la **topologia** del sistema (come sono collegati i componenti e come circolano segnali e informazioni);
- la **manutenzione** del sistema nel tempo;
- gli **operatori umani** coinvolti nel processo.

Tutti questi elementi contribuiscono alla sicurezza complessiva: trascurarne uno potrebbe compromettere l'obiettivo generale.

9.3 Pericolo (Hazard)

DEFINITION Pericolo / Hazard

Ciascuna situazione potenzialmente dannosa in cui il sistema può causare danno. Il modo più importante per migliorare la sicurezza di un sistema è **identificare tutte le modalità** in cui esso può causare danno. Esempio: in un macchinario industriale, una parte meccanica in movimento che può causare una lesione a un operatore.

9.3.1 Le due dimensioni del pericolo: Rischio e Danno

DEFINITION Rischio

La **probabilità** che il pericolo si trasformi effettivamente in un incidente e in un danno: la chance che una persona o un oggetto subiscano conseguenze a causa di quel pericolo, passando da una condizione potenziale a un evento reale. Un rischio elevato significa che è molto probabile che il pericolo si manifesti (perché presente frequentemente o perché i controlli sono scarsi); un rischio basso indica un evento raro.

DEFINITION Danno

La **gravità delle conseguenze** se il pericolo dovesse concretizzarsi. È una stima quantitativa o qualitativa: può variare da conseguenze lievi (danni minori, nessun ferito) fino a conseguenze gravissime o catastrofiche (lesioni gravi, perdita di vite umane, grandi danni materiali o ambientali).

EXAMPLE Banco di prova con braccio robotico

Il **pericolo** è il braccio robotico in movimento, che potrebbe colpire un operatore se si avvicina troppo. Il **rischio** dipende da quanto è probabile che qualcuno entri nella zona di movimento del robot mentre è in funzione (errore o disattenzione, o fallimento dei sistemi di rilevamento presenza). Il **danno** potenziale è la severità dell'infortunio in caso di impatto: da un livido fino a lesioni mortali, a seconda di forza e velocità del braccio.

Identificare pericoli come questo e valutarne rischio e danno viene effettuato redigendo un documento di progetto noto come **Risk Analysis**. Individuati i pericoli si progettano **contromisure** (sensori di presenza, barriere di sicurezza, procedure operative) per prevenire l'incidente o mitigarne le conseguenze.

9.3.2 Valutazione quantitativa e qualitativa

Per ogni pericolo identificato si valutano due aspetti: la **probabilità di occorrenza** e la **gravità del danno**. Spesso vengono suddivisi in categorie:

- probabilità: *frequente, occasionale, rara, estremamente improbabile*;
- conseguenze: da *lievi* (danni trascurabili) fino a *catastrofiche* (perdite umane o gravi danni materiali/ambientali).

9.3.3 Matrice dei rischi

Incrociando le categorie di probabilità e quelle di gravità del danno si rappresenta il rischio su una **matrice**: ogni combinazione corrisponde a un livello di pericolo più o meno critico. Un evento molto probabile con conseguenze gravi dà un rischio alto; un evento rarissimo con danno minimo dà un rischio basso. La matrice rischio-danno fornisce un quadro chiaro su quali pericoli richiedono interventi immediati e quali sono accettabili.

9.3.4 Classi di pericolo e accettabilità del rischio

- I. Inaccettabile** in qualsiasi circostanza: pericolo grave e probabile, da eliminare o ridurre assolutamente; non è ammissibile continuare l'operatività senza mitigazione;

- II. **Indesiderabile**: tollerabile solo se ogni ulteriore riduzione è impraticabile o i costi sarebbero enormemente sproporzionati rispetto ai benefici in termini di sicurezza;
- III. **Tollerabile** (accettabile): solo se il costo o l'impegno per ridurlo ulteriormente supera il miglioramento di sicurezza ottenuto; il pericolo è sotto controllo;
- IV. **Accettabile** così com'è, anche se potrebbe essere opportuno monitorarlo: pericolo così remoto o di danno così lieve da non richiedere misure correttive attive, ma tenuto sotto osservazione.

RISCHIO TOLLERABILE: CONSIDERAZIONI ETICHE

- **Valore della vita vs. analisi economica**: non ridurre il rischio al minimo possibile può compromettere la protezione della vita umana, anche se il miglioramento aggiuntivo appare economicamente sproporzionato;
- **Trasparenza e consenso informato**: informare pienamente le parti interessate sulle scelte e sui rischi residui, garantendo il diritto di essere consapevoli e coinvolti;
- **Responsabilità morale di progettisti e istituzioni**: oltre alla logica economica, adottare tutte le misure ragionevoli per minimizzare il rischio, in considerazione dei valori umani e sociali.

9.4 Sistemi di base per garantire la sicurezza

Identificati e valutati i pericoli, entrano in gioco i sistemi di protezione. Due meccanismi di base, spesso complementari:

1. **Interlock** (blocchi dinamici);
2. **Guard** (sbarramenti fisici).

9.4.1 Interlock (blocco)

DEFINITION Interlock

Sistema **attivo** che impedisce al sistema pericoloso di funzionare a meno che non siano soddisfatte determinate condizioni di sicurezza: monitora una condizione e **blocca l'operazione** se la situazione non è sicura.

Esempi:

- **Semaforo rosso ferroviario**: segnala che il tratto di binario successivo è occupato e di fatto impedisce (in combinazione con i sistemi di bordo e il macchinista) l'avanzamento del treno finché non diventa verde; il «blocco» viene annullato solo quando la condizione di sicurezza (binario libero) è soddisfatta;
- **Interlock software**: in robotica collaborativa, la funzione *safety monitored stop* ferma immediatamente il robot se un sensore rileva la presenza di un operatore umano troppo vicino.

In entrambi i casi l'interlock è **logico**: il treno non può avanzare finché il treno successivo non è distante; il robot non può continuare a muoversi finché la persona non esce dall'area pericolosa.

9.4.2 Guard (sbarramento)

DEFINITION Guard / sbarramento

Sistema **passivo**: barriera fisica o dispositivo che impedisce l'accesso di persone o altri sistemi a zone pericolose. Invece di agire sul funzionamento della macchina, agisce sulle persone o sulle cose, tenendole lontane dal pericolo.

Esempi: sbarre di un passaggio a livello (bloccano fisicamente la strada al passaggio del treno); gabbie protettive attorno ai robot industriali o ai macchinari (prevencono il contatto accidentale con l'area pericolosa).

9.4.3 Differenze e similitudini

- A differenza dell'interlock, la guardia **non ferma attivamente la macchina**: il macchinario potrebbe continuare a muoversi, ma grazie alla barriera nessuno può avvicinarsi abbastanza da farsi male;
- allo stesso tempo, limitare la libertà di movimento di un operatore ne limita l'operatività;
- i due approcci possono **coesistere**: ad esempio una porta di accesso a un'area robotizzata può essere chiusa da una guardia (serratura di sicurezza) e collegata a un interlock elettronico che spegne il robot se la porta viene aperta.

9.5 Ruolo dell'oggetto controllato

Aspetto cruciale della progettazione: valutare quanto l'oggetto o il processo controllato possa essere **messo in uno stato sicuro in caso di emergenza**. Se si verifica un pericolo, il sistema controllato può essere arrestato o reso innocuo in tempo? La risposta dipende dalla natura del sistema e determina come impostare le misure di sicurezza.

9.5.1 Pericoli non controllabili

Se l'oggetto controllato **non può** limitare o interrompere rapidamente la sua funzione pericolosa, allora il superamento di una guardia (violazione della barriera) costituisce un **pericolo non mitigabile**.

9.5.2 Pericoli controllabili

Se l'oggetto sotto controllo **può** ridurre, arrestare o mettere in sicurezza la propria funzione pericolosa, allora il superamento di una guardia può essere reso sicuro da un **interlock**: anche se una persona entra in una zona vietata, un meccanismo interviene automaticamente per neutralizzare il pericolo.

Esempi:

- **Robot collaborativo**: *Safety Monitored Stop, Speed and Separation Monitoring e Power and Force Limiting* non sono altro che interlock molto sofisticati;
- **Macchine utensili**: ripari che, se aperti, attivano interruttori di sicurezza (interlock elettromeccanici) che spengono la macchina all'istante. Esempio: una sega circolare industriale protetta da carter — se l'operatore rimuove il carter (superando la guardia fisica), un interruttore interrompe l'alimentazione e blocca la lama in frazioni di secondo.

Quando l'oggetto controllato è in grado di raggiungere rapidamente uno stato sicuro, interlock e guardie insieme forniscono **livelli ridondanti di protezione**: la guardia cerca di evitare l'errore umano limitando l'accesso, l'interlock interviene se tale errore avviene comunque.

9.6 Classificazione dei sistemi in base alla gestione dei guasti

I sistemi di controllo possono essere classificati in base alle strategie con cui rilevano e affrontano situazioni di malfunzionamento: *fail safe, fail operational, fail silent/fail explicit, bizantini*.

9.6.1 Sistemi Fail Safe

DEFINITION Fail-safe

Un sistema è fail-safe quando, in caso di guasto o malfunzionamento, è in grado di portare l'oggetto controllato in uno **stato di sicurezza** (o mantenerlo in sicurezza): se qualcosa va storto, il sistema «cade» in una condizione che non causa danni o li minimizza. Comportamento desiderabile in molti sistemi critici.

Esempi di meccanismi fail-safe:

- **Frenatura di emergenza di un ascensore**: se la fune di sollevamento si rompe, scattano automaticamente freni meccanici che bloccano la cabina, impedendo la caduta libera;
- **Valvola di sicurezza di una pentola a pressione**: se la pressione interna supera un certo limite, la valvola si apre e sfoga il vapore, evitando l'esplosione.

Il comportamento fail-safe spesso dipende dalle **caratteristiche intrinseche dell'oggetto controllato** e potrebbe non essere possibile in tutte le condizioni:

- un'automobile moderna in caso di guasto al motore permette in genere al conducente di accostare e fermarsi in sicurezza (fail-safe);
- un aeroplano non può «fermarsi in volo» se i motori si spengono: durante decollo o atterraggio la perdita di potenza può essere catastrofica perché non esiste uno stato di sicurezza facilmente raggiungibile in aria.

Se il raggiungimento di uno stato di sicurezza è fisicamente possibile, il sistema di controllo può essere progettato per facilitarlo: i treni hanno sistemi che inseriscono automaticamente la frenatura d'emergenza se il macchinista non risponde ai segnali di vigilanza (il cosiddetto «uomo morto»). **Fail-safe non significa che non avvenga mai un incidente**, ma che il sistema è progettato per minimizzare le conseguenze quando accade un guasto specifico.

9.6.2 Watchdog (sistema di sorveglianza)

DEFINITION Watchdog

Dispositivo o processo che **monitora costantemente** il corretto funzionamento di un componente principale, pronto a intervenire se qualcosa non va. È un sistema di controllo aggiuntivo, di solito molto semplice e affidabile, che serve a controllare il sistema di controllo primario. Nei sistemi complessi può essere necessario per garantire il comportamento fail-safe.

Il watchdog verifica periodicamente se il sistema principale è attivo e reattivo, tipicamente attraverso un segnale o una piccola interazione: il sistema principale deve «dare un cenno» a intervalli regolari. Se il watchdog non riceve conferma entro un certo intervallo, presume il guasto e attiva le azioni di emergenza (spesso portando il sistema in modalità fail-safe). Esempio: nelle metropolitane il macchinista deve premere un pulsante/pedale periodicamente; in caso contrario scatta la frenata automatica d'emergenza.

9.6.3 Sistemi Fail Operational

DEFINITION Fail-operational

Un sistema fail-operational è in grado di **continuare a funzionare nonostante un guasto**, garantendo comunque un livello di sicurezza accettabile. L'obiettivo non è spegnere tutto in sicurezza, ma mantenere attivo il servizio in modo sicuro, perché interromperlo potrebbe essere altrettanto rischioso o inaccettabile quanto il guasto stesso.

Si ottiene principalmente attraverso:

- la **ridondanza** dei componenti critici;
- un'efficace **rilevazione dei guasti** (meccanismo che si accorge del guasto e isola il componente difettoso).

DEFINITION Sistema degradato

Sistema che opera con uno o più componenti guasti ma funziona ancora: capacità ridotte o meno ridondanza, ma svolge comunque la sua funzione.

È fondamentale non mascherare completamente i guasti senza segnalare il degrado: se il sistema continua a funzionare come se nulla fosse, un ulteriore guasto può trovare il sistema già indebolito e portare al collasso. I sistemi fail-operational devono **tollerare i guasti e comunicare** che operano in modalità degradata.

EXAMPLE Aereo di linea

Anche se un motore si spegne in volo, l'aereo è progettato per continuare a volare con gli altri motori (a costo di ridurre l'altitudine o deviare; nota: dipende da diverse caratteristiche). Molte funzioni critiche

sono triplicate (tripli sistemi idraulici, tripli computer di controllo di volo, ecc.) così che il guasto di un singolo elemento non comprometta la missione.

EXAMPLE Centrale nucleare e sensoristica ridondante

Un sistema di pompe raffredda il nocciolo. Se una pompa si ferma (guasto), altre pompe devono entrare in azione immediatamente (fail-operational). Come rilevare che la pompa principale è ferma? Con un sensore di flusso o pressione. Ma anche il sensore può guastarsi: se non rileva nulla, la pompa è ferma o il sensore è rotto?

Si introduce la ridondanza nella sensoristica: due sensori indipendenti per la stessa grandezza. Ma se un sensore segnala «OK» e l'altro «guasto», a chi credere? Con due soli sensori, in caso di conflitto il sistema non sa quale informazione sia corretta.

DEFINITION TMR — Triple Modular Redundancy

Si utilizzano **tre sensori** per la stessa grandezza: in condizioni normali tutti e tre danno lo stesso responso; se uno differisce, i due che concordano «vincono» per **maggioranza**. La probabilità che due sensori su tre siano guasti contemporaneamente dando lo stesso valore sbagliato è estremamente bassa: se un sensore ha 1/1000 di probabilità di guasto in un dato intervallo, il doppio guasto indipendente è dell'ordine di 1/1.000.000.

Nel **sistema di votazione** (*voting*) i sensori «votano» sullo stato del componente monitorato e la maggioranza determina la decisione (es. spegnere la pompa o attivare la riserva se due sensori su tre rilevano il guasto della principale). In ambito aeronautico e spaziale la tripla ridondanza è comune nei sistemi di guida e controllo: i computer di bordo sono triplicati e confrontano continuamente i risultati; se uno «impazzisce», gli altri due lo escludono.

9.6.4 Sistemi Fail Silent e Fail Explicit

DEFINITION Fail-silent

Un sistema fail-silent, in caso di guasto, **smette di funzionare senza segnalare alcun allarme** o indicazione esplicita: da fuori non si vede nulla di diverso, semplicemente non risponde più. Comportamento rischioso: altri componenti o operatori potrebbero non distinguere un malfunzionamento da un semplice ritardo.

Per ovviare ai problemi del fail-silent si trasforma il guasto silenzioso in un guasto esplicito (**fail-explicit**). La soluzione generale è **monitorare il sistema attivo e impostare dei timeout**: se un componente A invia una richiesta a B, parte contemporaneamente un timer tarato sul tempo massimo di risposta atteso (*worst-case*); se il timer scade prima della risposta, A dichiara B guasto e intraprende azioni di sicurezza. Meglio un sistema fail-explicit, con guasto dichiarato apertamente, che un silenzio ambiguo.

EXAMPLE Semafori stradali

Se il controllore elettronico del semaforo si guasta, le luci entrano in modalità lampeggiante (giallo lampeggiante) o si spengono in modo riconoscibile, segnalando agli automobilisti di procedere con cautela come a uno stop. Comportamento preferibile a un semaforo bloccato su verde o rosso fisso: in quel caso gli utenti potrebbero non capire il guasto, con confusione o pericolo.

9.6.5 Sistemi Bizantini

DEFINITION Guasto bizantino

Comportamento **arbitrariamente errato o malizioso** di un componente del sistema, dal problema dei «Generali di Bisanzio» dell'informatica distribuita. Un componente bizantino può fornire informazioni

false, contraddittorie o deliberatamente fuorvianti nel peggiore modo possibile: può fingere di funzionare inviando comandi sbagliati, funzionare a intermittenza, o comportarsi diversamente con componenti differenti. È il caso più difficile da gestire: il componente sembra «vivo» ma non è affidabile, anzi può ingannare attivamente il sistema.

EXAMPLE Centralina sabotata

In un sistema di controllo distribuito con più centraline, una centralina sabotata da codice maligno (attacco hacker) potrebbe inviare letture di sensori falsificate o comandi pericolosi agli attuatori. Dal punto di vista degli altri componenti è imprevedibile: a volte risponde correttamente, altre no, oppure fornisce dati incoerenti.

Le tecniche di tolleranza ai guasti bizantini aggiungono ridondanza e meccanismi di controllo incrociato tali che, anche se un nodo si comporta male, il sistema possa identificarlo e isolarlo continuando a funzionare correttamente. Spesso si assume un limite (es. al massimo 1 nodo bizantino alla volta), perché tollerare guasti arbitrari simultanei multipli diventa molto complesso.

Approccio Voltan

L'approccio Voltan trasforma potenziali comportamenti bizantini in comportamenti **fail-silent** attraverso duplicazione e confronto costante tra due sottosistemi gemelli:

- ogni nodo critico è realizzato da **due processori identici** che eseguono esattamente gli stessi calcoli in parallelo, in modo quasi sincrono, comunicando attraverso un canale hardware dedicato ad alta velocità;
- ogni ingresso viene fornito simultaneamente a entrambi; ciascuno elabora indipendentemente con lo stesso algoritmo deterministico;
- a fine elaborazione i due gemelli **confrontano i risultati** attraverso il canale diretto.

Se i risultati coincidono: entrambi funzionano correttamente; l'output viene considerato valido e marcato con **due «firme» digitali**, una di ciascun processore, così i nodi riceventi sanno che è stato approvato da entrambi (impedisce a un singolo gemello bizantino di inviare messaggi da solo).

Se i risultati differiscono: qualcosa non va (un processore è guasto o attaccato); il nodo si **blocca immediatamente**, non trasmette alcun output e non firma più messaggi, entrando in uno stato di blocco definitivo.

Formalmente: ciascun gemello riceve la richiesta, elabora una risposta e la invia al nodo gemello. Due casi:

- A. La risposta del gemello è identica a quella elaborata in proprio: il nodo N firma la sua risposta e la invia;
- B. La risposta del gemello differisce: N si blocca indefinitamente senza formare la propria risposta.

L'effetto pratico: anche se uno dei due processori si comporta in modo bizantino, il nodo nel complesso si comporta come **fail-silent**: o dà risposte corrette (risultati coincidenti), oppure tace completamente bloccandosi (gemelli in disaccordo). Si assume che la probabilità di doppio guasto bizantino simultaneo sia trascurabile: almeno uno dei due sarà onesto e qualunque disallineamento farà scattare il blocco. Si controlla così il caso peggiore, trasformando un problema imprevedibile (guasto sleale) in uno più gestibile (da fail-silent a fail-explicit con timeout).

SISTEMI FAULT TOLERANT AVANZATI

L'approccio Voltan rientra nei sistemi fault tolerant avanzati, ispirati al Problema dei Generali Bizantini. Nella pratica moderna duplicazione e confronto sono usati ad esempio nelle centraline automobilistiche per funzioni critiche (ABS, guida autonoma): due microcontrollori calcolano in parallelo e verificano la reciproca coerenza. Anche se i guasti bizantini sono rari nell'hardware puro, questa categoria concettuale prepara ad affrontare sabotaggi intenzionali ed errori non convenzionali con il massimo rigore.

BIBLIOGRAFIA

Kopetz H., *Real-Time Systems, Design Principles for Distributed Real-Time Applications*, Kluwer Academic Publisher, 1997; Storey N., *Safety Critical Computer Systems*, Pearson Prentice Hall, 1996; Leveson

N.G., *Engineering a Safer World*, MIT Press.

Capitolo 10

Affidabilità, sicurezza e Fault Analysis: FMEA e FTA

10.1 Fault, Error e Failure: definizioni e differenze

Nel contesto dell'affidabilità dei sistemi, i termini *fault*, *error* e *failure* descrivono concetti distinti lungo la **catena causale** di un malfunzionamento. Attenzione: i termini possono denotare concetti differenti in standard di sicurezza differenti (qui si seguono IEC 60050 e IEC 61508).

DEFINITION Fault — guasto o difetto

Difetto o malfunzionamento **interno a un componente** del sistema (hardware o software). È lo stato di un componente quando ha cessato di funzionare correttamente o contiene un difetto latente. Esempi: cortocircuito in un circuito elettrico, transistor fisicamente rotto, bug nel codice di un software (difetto di programmazione).

DEFINITION Error — errore

Deviazione dallo stato o comportamento atteso del sistema, causata tipicamente dall'attivazione di un fault. È una condizione anomala all'interno del sistema (per esempio un valore di calcolo errato in un software, o un sensore che fornisce una lettura sbagliata) risultante da un fault attivo. Importante: un fault può rimanere **latente** senza generare immediatamente errori, finché non viene attivato in condizioni particolari.

DEFINITION Failure — mancata funzionamento, avaria

Incapacità del sistema di svolgere la funzione richiesta o di rispettare i requisiti specificati. La failure è osservabile a livello di sistema (al suo confine esterno) ed è l'evento finale in cui il comportamento erogato non corrisponde a quello previsto. Si manifesta quando un errore non viene gestito appropriatamente e provoca un malfunzionamento visibile all'utente o all'ambiente operativo.

10.1.1 Catena Fault–Error–Failure

Un **fault** attivo può provocare uno o più **error**, i quali a loro volta possono propagarsi e causare una **failure** del sistema:

fault (causa radice) $\longrightarrow$ error (stato incorretto) $\longrightarrow$ failure (effetto finale osservabile)

La failure è l'effetto finale osservabile (ad esempio l'interruzione di servizio o un incidente).

INTUITION Failure come proprietà sistemica emergente

Non tutti i fault portano necessariamente a una failure: un sistema con ridondanza può tollerare un guasto senza esibire errori visibili (il fault resta nascosto o latente e il sistema funziona grazie ai meccanismi

di backup). Viceversa, non tutte le failure sono dovute a un fault hardware attivo: il sistema può fallire anche senza componenti guasti, per un requisito errato di progettazione o una condizione non prevista. La failure è quindi una **proprietà sistemica emergente**: un sistema può «funzionare male» anche se ogni singolo componente funziona secondo specifica, quando la combinazione di componenti, ambiente operativo e input porta a un comportamento non sicuro.

10.1.2 Esempi nei vari ambiti

EXAMPLE Ambito industriale

In una linea di produzione automatizzata, un **fault** è la rottura di un sensore di prossimità su un nastro trasportatore (guasto hardware interno). Finché il pezzo è correttamente rilevato da un sensore ridondante, il sistema non mostra anomalie: il fault rimane latente. In assenza di ridondanza, quando un oggetto non viene rilevato dal sensore guasto, il sistema entra in uno stato incoerente (**error**): il PLC non «vede» il pezzo presente quando invece c'è. L'errore può propagarsi causando una **failure**: la macchina non esegue la lavorazione sul pezzo (funzione richiesta mancata) e la linea si ferma con blocco produttivo.

EXAMPLE Ambito informatico/software

In un software gestionale, un **fault** è un bug nel codice (difetto di programmazione). Resta latente se la porzione di codice difettosa non viene eseguita; quando il programma esegue quella funzione, il bug si attiva generando un calcolo errato (es. overflow o risultato logicamente sbagliato): è l'**error** (stato interno incorretto). Se l'errore non viene intercettato, porta a una **failure** visibile: terminazione anomala oppure output sbagliato all'utente. In sintesi: fault = bug nel codice; error = stato inconsistente (variabile fuori range); failure = il software non svolge il servizio atteso (report con dati errati o crash).

EXAMPLE Ambito aerospaziale

Un **fault** possibile è il guasto di un altimetro radar di un aeromobile. Questo fault hardware provoca un **error** nei dati di quota inviati al computer di volo (altitudine letta sballata di parecchi metri). Se l'aereo dispone di un sistema di votazione tripla (tripla ridondanza, Capitolo 9), l'errore può essere mascherato confrontando le tre misure e scartando quella anomala; in caso contrario può condurre a una **failure** grave: il pilota automatico potrebbe mantenere una quota errata.

Altri scenari simili:

- **Automotive**: fault = cedimento di un tubo dei freni (guasto meccanico) → error = pressione idraulica nulla → failure = perdita della capacità frenante;
- **Sanitario**: fault = malfunzionamento di una pompa di infusione → error = dosaggio errato del farmaco → failure = rischio per il paziente.

10.2 Relazione tra Affidabilità e Sicurezza

Affidabilità e sicurezza sono due proprietà chiave, strettamente correlate ma **non equivalenti**:

- **Affidabilità (reliability)**: probabilità che un sistema compia la funzione prevista *senza guasti* per un intervallo di tempo definito e in date condizioni operative;
- **Sicurezza (safety)**: proprietà di un sistema di *non causare danni inaccettabili* a persone, beni o ambiente; spesso definita come «libertà dal rischio inaccettabile di danni».

10.2.1 Differenze concettuali

- L'affidabilità riguarda la **continuità di servizio** e l'assenza di failure *tout-court*;
- la sicurezza si concentra sull'**assenza di conseguenze catastrofiche o pericolose**;

- un sistema affidabile funziona senza guasti frequenti; un sistema sicuro evita situazioni di pericolo *anche in presenza di guasti*.

10.2.2 Conseguenze

- Un sistema può essere molto affidabile ma **non sicuro**;
- al contrario, un sistema può essere molto sicuro ma **non particolarmente affidabile**.

EXAMPLE Macchinario molto sicuro ma poco affidabile

Si consideri un macchinario industriale dotato di interblocchi e arresti di emergenza estremamente cautelativi: ogni minima anomalia fa scattare un arresto immediato (fail-safe), prevenendo il rischio di infortuni. Il macchinario è *sicuro*, perché in presenza di potenziali problemi si ferma in modalità sicura. Tuttavia i frequenti fermi lo rendono poco disponibile e affidabile dal punto di vista produttivo (bassa continuità di servizio): la sicurezza è garantita a scapito dell'affidabilità operativa.

IN SINTESI

L'affidabilità mira all'assenza di guasti in generale; la sicurezza mira all'assenza di conseguenze inaccettabili. Un progetto ben fatto deve perseguire **entrambe**: sistema funzionante in modo continuo e, se avvengono guasti, senza mettere in pericolo persone o obiettivi fondamentali.

10.3 Analisi dei guasti: FMEA e FTA

Per gestire e migliorare sia l'affidabilità che la sicurezza, l'ingegneria si avvale di metodologie analitiche strutturate. Le due più diffuse:

- **FMEA** (Failure Mode and Effects Analysis): approccio **bottom-up** (induttivo);
- **FTA** (Fault Tree Analysis): approccio **top-down** (deduttivo).

Nate in ambiti industriali diversi, oggi fanno parte delle best practice in molti settori, con standard dedicati (IEC 60812 per FMEA, IEC 61025 per FTA).

10.3.1 FMEA — Failure Modes and Effects Analysis

DEFINITION FMEA

Tecnica di analisi **induttiva, dal basso verso l'alto**: si esaminano sistematicamente i potenziali **modi di guasto** (*failure modes*) di ciascun componente o sottosistema e si valutano gli **effetti** di tali guasti sul sistema complessivo. Obiettivo: identificare cosa potrebbe andare storto in ogni parte del sistema, comprendere le conseguenze di ogni guasto e classificare i guasti in base alla **criticità**, per mitigare i rischi prima che si manifestino.

In pratica la FMEA aiuta i progettisti a chiedersi: «qual è ogni possibile modo in cui ogni elemento può guastarsi, e cosa succede al sistema se ciò accade?».

LIMITAZIONI E BEST PRACTICE DELLA FMEA

- È focalizzata su **guasti singoli**: non considera facilmente eventi combinati o cascata di guasti simultanei;
- può richiedere un enorme **sforzo documentale**: si analizzano anche guasti con probabilità remota o effetti trascurabili, investendo tempo su aspetti non critici;
- best practice: applicare la FMEA in modo **mirato ai sottosistemi critici**, magari integrandola con altre analisi.

10.3.2 FTA — Fault Tree Analysis

DEFINITION FTA

Tecnica complementare alla FMEA, con approccio **deduttivo dall'alto verso il basso** (*top-down*). Invece di partire dai componenti, parte da un **evento indesiderato apicale** (*top event*) — tipicamente una failure o un incidente — e **risale** alle possibili cause prime attraverso un diagramma logico ad albero.

Come si esegue una FTA:

- si costruisce un albero dei guasti in cui il **nodo radice** è il problema da evitare (es. «pericolo di incendio in macchina X» o «perdita totale di potenza») e i **rami** rappresentano combinazioni di eventi di guasto che possono condurre al top event;
- si utilizzano **operatori logici**:
 - porte **OR** (alternative): eventi che *individualmente* possono causare l'evento superiore;
 - porte **AND** (concomitanza): condizioni che devono verificarsi *tutte insieme* perché avvenga l'evento superiore;
- il risultato è un modello grafico che descrive tutte le combinazioni di guasti di base che portano alla failure considerata.

10.3.3 Esempio ed esercizio: laser industriale con interblocco

EXAMPLE Sistema laser

Sistema laser controllato da un computer attraverso un attuatore di potenza (*power driver*), che può essere attivato solo se un dispositivo di protezione (*copertura*) viene chiuso. **Evento pericoloso**: il laser si attiva a copertura alzata.

Possibile albero di guasto:

- **Top Event**: «Il laser si attiva con la protezione aperta»;
- causa immediata: «mancata rilevazione della protezione aperta» **AND** «comando inavvertito di accensione laser» — entrambe *necessarie* per il pericolo (porta AND);
- «mancata rilevazione protezione» può derivare da un **OR** di: sensore di posizione guasto OR errore umano (interruttore bypassato);
- «comando inavvertito» può derivare da: guasto al controller OR errore software.

BIBLIOGRAFIA

Kopetz H., *Real-Time Systems*, Kluwer, 1997; Storey N., *Safety Critical Computer Systems*, Pearson Prentice Hall, 1996; Leveson N.G., *Engineering a Safer World*, MIT Press.

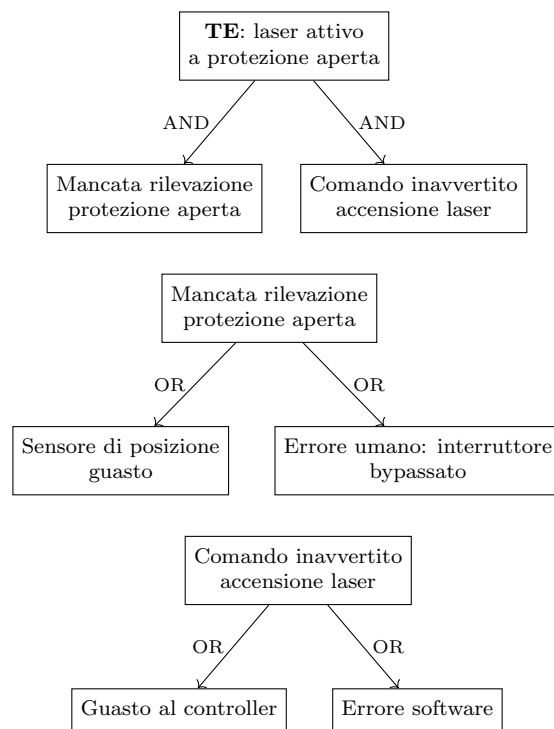


Figura 10.1: FTA per il sistema laser: il top event richiede *entrambe* le cause immediate (porta AND); ciascuna può derivare da alternative indipendenti (porte OR). L'albero evidenzia che per avere il laser acceso a protezione aperta occorre sia il guasto/errore sul sensore sia un comando di attivazione errato, simultaneamente.

Capitolo 11

Normative sulla sicurezza funzionale nei sistemi di controllo

11.1 Obiettivo delle norme

Assegnare **parametri quantitativi** ai sistemi che assolvono funzioni di sicurezza non svolte dal sistema di controllo ordinario (analogia con il watchdog), per esempio:

- Funzione di Arresto di Emergenza (E-Stop);
- Funzione di Controllo Interblocco di un Riparo/Guardia;
- Funzione di controllo operazione a due mani.

Attraverso tali parametri si stima la congruità e l'efficacia di un sistema di sicurezza in relazione a un caso d'uso dato, rendendo i risultati **verificabili** secondo metodologie stabilite da norme internazionali.

11.2 Terminologia

DEFINITION Sicurezza funzionale

Implementata da sistemi che **monitorano** e, in caso di situazioni pericolose, **assumono il controllo della macchina** al fine di garantire un funzionamento sicuro.

DEFINITION SRCS — Safety-Related Control System

Sistema di controllo relativo alla sicurezza: implementa le funzioni di controllo relative alla sicurezza (*SRCF*, Safety-Related Control Functions) rilevando le condizioni pericolose e portando il sistema sotto controllo in uno stato **fail safe**, garantendo che venga eseguita un'azione specifica.

La norma **EN ISO 13849-1** definisce le parti di un sistema di controllo di una macchina che servono per realizzare le funzioni di sicurezza come **SRP/CS** (*Safety-Related Parts of Control Systems*).

11.3 Le due norme: IEC 62061 e ISO 13849-1

- **IEC 62061** (deriva da IEC 61508): prescrive standard per sistemi tesi alla riduzione dei rischi in sistemi **E/E/EP** (elettrici, elettronici ed elettronici programmabili);
- **ISO 13849-1**: norma soprattutto l'impiego, sia in ambito meccanico che idraulico e pneumatico, di componenti elettrici ed elettronici programmabili e lo sviluppo della mecatronica nell'ambito industriale.

Nota: ISO non è un acronimo (in inglese: International Organization for Standardization); deriva dal greco *isos* che significa «uguale». Entrambi gli standard forniscono linee guida per l'implementazione della sicurezza funzionale.

11.4 Ruolo del controllo nella sicurezza funzionale

La distinzione fondamentale è tra:

- **controllo ordinario:** governa il funzionamento della macchina (es. regola la velocità di un nastro trasportatore);
- **controllo di sicurezza:** sorveglia le condizioni di rischio e interviene quando necessario (es. arresta il nastro se viene aperto un riparo mobile).

La sicurezza funzionale non è affidata alla sola logica di comando ordinaria, ma a funzioni e componenti **dedicati** alla sicurezza. Tali elementi possono essere **integrati** nel sistema di controllo oppure **separati** da esso, ma hanno sempre il compito di:

1. rilevare condizioni pericolose;
2. prevalere sul controllo ordinario, se necessario;
3. condurre la macchina in uno stato sicuro (rimozione della coppia al motore, arresto del movimento, blocco del riavvio automatico).

PLC DI SICUREZZA VS. RELÈ DI SICUREZZA

PLC di sicurezza integrato nel sistema: gestisce logiche di sicurezza complesse; si integra con il controllo della macchina; indicato per impianti articolati e funzioni multiple.

Relè di sicurezza dedicato: realizza funzioni di sicurezza specifiche; struttura più semplice e meno flessibile; indicato per applicazioni con esigenze di sicurezza circoscritte.

INTUITION Perché non basta un PLC ordinario

Il controllo ordinario impedisce le situazioni pericolose *previste dalla logica di esercizio*; la sicurezza funzionale interviene quando si verificano **guasti, anomalie o perdita del controllo corretto**. Non basta comandare correttamente la macchina: occorre garantire il raggiungimento di uno stato sicuro in caso di malfunzionamento. Esempio: in un incrocio semaforico la logica ordinaria evita il verde simultaneo su direttrici incompatibili; la sicurezza funzionale richiede che, in caso di guasto del controllo, il sistema si porti in uno stato fail safe.

11.5 Principi generali nella riduzione del rischio

1. **Eliminare o minimizzare** ogni possibile rischio a partire dal progetto del dispositivo;
2. applicare le necessarie **misure di protezione** verso i pericoli che non possono essere eliminati (sicurezza funzionale);
3. **informare gli utenti** dei rischi che non possono essere eliminati nonostante tutte le misure protettive.

11.6 Guasto pericoloso e random fault

DEFINITION Guasto pericoloso (random fault)

«Guasto che pregiudichi la funzione di controllo relativa alla sicurezza (SRCF) come conseguenza di guasti hardware casuali». Un *random fault* è un guasto casuale nel tempo, dovuto per esempio a: usura dei componenti, invecchiamento, variazioni fisiche o ambientali, difetti hardware che si manifestano durante l'esercizio. Non è un errore «voluto» o sistematico di progetto, ma un guasto che può comparire in modo **statisticamente descrivibile**.

CURVA A VASCA DA BAGNO E VITA UTILE

Il tasso di guasto λ (numero di guasti per ora) rimane approssimativamente costante per un certo periodo

di tempo ($T_u - T_a$) detto **vita utile**. Il tempo M è detto **vita media**. Un po' prima della fine della vita utile si sostituiscono preventivamente tutti i componenti, anche se ancora funzionanti. Esempio: il motore di un aereo da turismo viene sostituito o revisionato radicalmente attorno alle 1200 ore.

11.7 Valutazione delle prestazioni delle funzioni di sicurezza

Le due norme adottano parametri diversi ma equivalenti nello spirito:

Norma	Parametro	Significato
IEC 62061	SIL (Safety Integrity Level)	Livello di integrità della sicurezza
ISO 13849-1	PL (Performance Level)	Livello di prestazione

In entrambi i casi la valutazione quantitativa è espressa in **intervalli di probabilità** in cui si possono verificare guasti pericolosi in un'ora:

- possono essere espressi equivalentemente in intervalli temporali in cui può avvenire un guasto pericoloso;
- intervalli ampi indicano minor rischio di guasto pericoloso e viceversa;
- le prestazioni devono essere **adeguate al caso in esame**: il caso riceve prima una valutazione **qualitativa** della prestazione di sicurezza *richiesta*, a cui segue un progetto i cui parametri dovranno soddisfare quantitativamente tale valutazione (prestazione *ottenuta*).

11.7.1 Metodologia: PL/SIL richiesto e ottenuto

1. Si valuta il pericolo del caso sotto esame con una misurazione **qualitativa a priori**: si determina il PL o SIL **richiesto** alla funzione di sicurezza;
2. in base al PL/SIL richiesto si effettuano *a posteriori* scelte di progetto sul dispositivo, che portano a una funzione di sicurezza il cui PL/SIL **ottenuto** viene calcolato con metodi quantitativi;
3. il PL o SIL **ottenuto deve essere maggiore o uguale** a quello richiesto ($PL \geq PL_r$).

11.7.2 SIL (IEC 62061)

Il **SIL** determina il grado di affidabilità richiesto a una *SRCF* ed è espresso sulla base di una scala di probabilità oraria di verificarsi di un guasto pericoloso, quantificata dal **PFH_D** (*Probability of dangerous Failure per Hour*):

- esempio: SIL 1 significa che un guasto pericoloso può avvenire in un intervallo compreso tra centomila e un milione di ore;
- $PFH_D = 10^{-6} \Rightarrow 1$ guasto ogni 10^6 ore (pari a circa 10 anni);
- quindi SIL 1 corrisponde a un guasto pericoloso possibile in un intervallo tra 1 anno e 10 anni.

11.7.3 PL (ISO 13849-1)

Il **PL** è la capacità di un sistema di protezione di realizzare una funzione di sicurezza; è definito in base alla probabilità di guasto pericoloso come conseguenza di guasti hardware casuali potenzialmente pericolosi. Tale probabilità dipende da diversi parametri, tra cui il **MTTF_d** (Mean Time To dangerous Failure), che dipende dall'architettura del sistema (o sottosistema) e dalla stima dello stesso parametro per i singoli componenti.

11.7.4 Determinazione qualitativa del PL richiesto (PL_r)

1. Analisi e valutazione qualitativa del rischio associato alla funzione di sicurezza;
2. per ciascun rischio viene assegnato il **PL_r** (PL richiesto) attraverso un processo decisionale qualitativo (grafico normativo basato su: gravità delle conseguenze, frequenza ed esposizione al pericolo, possibilità di evitare il pericolo).

11.7.5 Determinazione qualitativa del SIL richiesto

In base alla **somma dei punteggi** di durata, probabilità ed evitabilità, assegnati per un evento pericoloso, si determina la **classe**; successivamente, dall'intersezione fra la colonna «conseguenze» e la colonna «classe», si ricava in maniera qualitativa il **SIL** da assegnare alla funzione di sicurezza. «OM» sta per «altre misure» (*other measures*) da utilizzare a livello di sistema.

11.7.6 Corrispondenza tra PL e SIL

I due parametri sono correlati: tabelle normative mettono in corrispondenza i livelli PL (da *a* a *e*) con i livelli SIL (1–3), essendo entrambi basati su intervalli di PFH_D.

11.8 Implementazione della funzione di sicurezza

Il passo successivo consiste nel definire la parte del sistema di controllo che implementa la funzione di sicurezza (SRP/CS) e, una volta definito tale sottosistema, determinarne:

- la **Categoria**, ovvero l'architettura (in serie, in parallelo/ridondante, con struttura diagnostica, ecc.);
- il **MTTF_D**;
- il **DC** (diagnostic coverage);
- il **CCF** (guasti da causa comune: derivano da un unico evento e non dipendono gli uni dagli altri; esempio: un fulmine mette fuori uso sia il sensore di contatto sia il sistema che lo controlla).

11.8.1 MTBF_d: tempo tra due guasti pericolosi

Il **MTTF_d** (Mean Time To dangerous Failure) è il parametro principale che definisce l'affidabilità di un componente o sistema. Nei contesti produttivi si usano componenti sostituibili o riparabili, quindi interessa la distanza temporale tra due guasti:

$$\text{MTBF}_d = \text{MTTF}_d + \text{MTTR}$$

dove **MTTR** (Mean Time To Repair) è il valore atteso del tempo di ripristino della funzionalità (riparazione o sostituzione). Quando può essere trascurato: $\text{MTBF}_d \cong \text{MTTF}_d$.

11.8.2 DC — Diagnostic Coverage

La copertura diagnostica è il rapporto fra la frequenza dei guasti pericolosi **rilevati** λ_{dd} e la frequenza dei guasti pericolosi **totali** λ_d :

$$DC = \frac{\lambda_{dd}}{\lambda_d}$$

11.8.3 Categorie (architetture)

Le categorie sono associate ad architetture con differente:

1. ridondanza;
2. comportamento al guasto;
3. requisiti richiesti;
4. principi per raggiungere la sicurezza funzionale (e in generale il comportamento della parte che implementano).

Utilizzando sistemi con categoria più alta è possibile raggiungere livelli di sicurezza più alti in termini di probabilità di guasto.

Architetture IEC 62061

Architettura A — tolleranza zero ai guasti, nessuna diagnostica. Blocchi in **serie logica**: il guasto di uno solo determina la perdita della funzione di sicurezza. Se si conoscono i tassi di guasto pericoloso di ogni elemento, è possibile determinare il tasso di guasto pericoloso del sottosistema.

Architettura B — tolleranza singola ai guasti, nessuna diagnostica. Due blocchi in **parallelo logico**: la perdita della funzione di sicurezza si ha solo nel caso di guasto di entrambi i blocchi (attenzione al CCF, Common Cause Failure).

Architettura C — tolleranza zero ai guasti, con diagnostica. Blocchi in serie logica, ma è inserito un **blocco diagnostico** (eventualmente integrato a livello del singolo sottosistema) che, rilevato il guasto, attiva una reazione successiva all'avaria: garantisce un intervento tempestivo ed evita la perdita della funzione di sicurezza.

Architettura D — tolleranza singola ai guasti, con diagnostica. Due blocchi in parallelo logico con **blocco diagnostico** che, rilevati guasti, attiva opportune azioni correttive.

Categorie ISO 13849-1

Categoria B. Architettura a **canale singolo** composta da: blocco di Input o sensori (blocco I), blocco di logica per l'elaborazione dei dati (blocco L), blocco di Output o attuazione (blocco O), un canale che soddisfa **requisiti minimi** di affidabilità (un guasto può portare alla perdita della funzione di sicurezza). Nessuna copertura diagnostica (DC); $MTTF_d$ da basso a medio; CCF non applicabile.

Categoria 1. Simile alla categoria B ma con $MTTF_d$ **più alto**.

Categoria 2. Canale singolo con aggiunto un **ramo di test** della funzione di sicurezza: verifica effettuata ad opportuni intervalli di tempo, nei momenti e nelle fasi più significative (avvio, esecuzione di movimenti e procedure pericolose). Quando il guasto è rilevato, il sistema porta la macchina in condizione sicura mantenuta finché il guasto non è eliminato. Vincoli importanti: (1) frequenza di richiesta dell'azione di sicurezza pari a 1/100 della frequenza di test; (2) $MTTF_d$ del canale di test maggiore della metà del $MTTF_d$ del canale della funzione.

Categorie 3 e 4. Architetture a **canale doppio**: le unità logiche dei due canali sono dotate di funzione di diagnostica, effettuata sia sulla base di segnali di monitoraggio sia per mezzo dell'analisi dei dati scambiati. **Categoria 3**: un singolo guasto non deve portare alla perdita della funzione di sicurezza e deve essere possibilmente rivelato. **Categoria 4**: un singolo guasto deve essere rivelato in occasione della richiesta della funzione di sicurezza o prima della richiesta successiva.

11.8.4 Relazione tra Categorie, DC, $MTTF_d$ e PL

Il grafico normativo (EN ISO 13849-1) mostra che il PL ottenibile dipende dalla combinazione di **categoria**, DC_{avg} e $MTTF_d$:

- la **categoria** descrive l'architettura del sistema di controllo di sicurezza: strutture più robuste consentono PL più elevati;
- il DC_{avg} esprime la capacità di rilevare i guasti pericolosi: all'aumentare della copertura diagnostica aumenta il PL ottenibile;
- il $MTTF_d$ rappresenta l'affidabilità dei componenti rispetto ai guasti pericolosi: componenti più affidabili, in un'architettura adeguata, permettono PL più alti.

11.9 Acronimi

- **IEC**: International Electrotechnical Commission;
- **ISO**: International Organization for Standardization (ma anche *isos*, uguale);
- **UNI**: Ente nazionale italiano di unificazione;
- **EN**: norme elaborate dal CEN (Comité Européen de Normalisation), Organismo di Normazione Europea. I Paesi membri CEN devono obbligatoriamente recepire le norme EN (in Italia diventano UNI EN).

BIBLIOGRAFIA

Sicurezza funzionale dei sistemi di controllo delle macchine: requisiti ed evoluzione della normativa, INAIL, 2013; EN 954-1:1996; EN ISO 13849-1:2006 + EC1:2009; EN IEC 62061:2005 (CEI 44-16).

Capitolo 12

Programmazione di funzioni di sicurezza in ST: casi di studio

12.1 Introduzione

La sicurezza funzionale nelle macchine è regolata da standard come **IEC 62061** (derivato da IEC 61508, focalizzato sui SIL) e **ISO 13849-1** (che introduce i PL, con categorie ereditate da EN 954-1), visti nel Capitolo 11. In entrambi gli standard la probabilità di guasto pericoloso di una funzione di sicurezza viene quantificata in termini di intervalli di PFH_D . Ad esempio, **PL e** (il livello più alto di ISO 13849-1) corrisponde a un intervallo di PFH_D molto basso (circa 10^{-8} – 10^{-7} per ora) ed è equivalente, in termini di affidabilità richiesta, a un **SIL 3**.

Una funzione di sicurezza deve raggiungere almeno il PL o SIL **richiesto** (PL_r o SIL_r) emerso dall'analisi dei rischi, e questo va dimostrato tramite calcoli quantitativi (MTTF_d , PFH_D , ecc.) dopo aver progettato l'architettura di sicurezza appropriata.

Nell'ambiente **CODESYS** (piattaforma IEC 61131-3) è possibile programmare funzioni di sicurezza in Structured Text sfruttando PLC di sicurezza.

12.2 Esempio 1: Funzione di Arresto di Emergenza (E-Stop)

12.2.1 Requisiti normativi

Una funzione di arresto di emergenza serve a arrestare rapidamente un macchinario in caso di pericolo imminente, tramite uno o più **pulsanti a fungo** di emergenza distribuiti sulla macchina. Secondo la norma **ISO 13850**, il dispositivo di E-Stop deve essere:

- facilmente accessibile;
- capace di provocare l'arresto del processo pericoloso **nel tempo più breve possibile**, senza creare rischi supplementari.

Dal punto di vista elettrico, l'arresto di emergenza può essere implementato come:

- **Stop di Categoria 0**: arresto immediato mediante rimozione dell'energia;
- **Stop di Categoria 1**: arresto controllato seguito da rimozione dell'energia.

Nell'esempio si considera uno stop di Categoria 0 che apre immediatamente il circuito di potenza del motore.

12.2.2 Struttura I–L–O della funzione

- **Sensore (I, Input)**: pulsante di emergenza tipicamente con contatti **NC** (normalmente chiusi) collegati a **due ingressi di sicurezza separati** nel PLC (canale doppio) per tollerare eventuali guasti;
- **PLC (L, Logica)**: nel codice ST si mappano gli ingressi a variabili booleane e si comanda un'uscita di sicurezza verso il contattore;
- **Attuatore (O, Output)**: per fermare un motore in modo sicuro la funzione può agire su due dispositivi:
 1. **contattore di sicurezza**;
 2. **ingresso STO** (Safe Torque Off) di un inverter, che toglie alimentazione al motore.

12.2.3 Programma ST

```

PROGRAM Safety_EStop
VAR
  EStop_Ch1 AT %IO.0 : BOOL; // Canale 1 pulsante E-Stop (NC, TRUE = non premuto)
  EStop_Ch2 AT %IO.1 : BOOL; // Canale 2 pulsante E-Stop (NC)
  ResetButton AT %IO.2 : BOOL; // Pulsante di reset manuale
  SafeOut AT %Q0.0 : BOOL; // Uscita di sicurezza verso contattore motore
  EStop_Latch : BOOL := FALSE; // Latch di stato di emergenza attivo
END_VAR

// Rilevazione comando di E-Stop
IF (NOT EStop_Ch1) OR (NOT EStop_Ch2) THEN
  SafeOut := FALSE; // disattiva immediatamente uscita sicura
  EStop_Latch := TRUE; // entra nello stato di arresto di emergenza latched
END_IF;

// Mantiene il motore fermo finché l'emergenza non viene resettata
IF EStop_Latch THEN
  SafeOut := FALSE;
END_IF;

// Reset manuale: pulsante rilasciato (entrambi i canali chiusi) e reset premuto
IF EStop_Latch AND EStop_Ch1 AND EStop_Ch2 AND ResetButton THEN
  EStop_Latch := FALSE; // esce dallo stato di emergenza
  SafeOut := TRUE; // riabilita uscita di sicurezza
END_IF;

```

MAPPATURA DEI SEGNALI: AT %I E AT %Q

AT %IO.n indica il **morsetto di ingresso** del PLC a cui è collegata la variabile; AT %Q0.n indica il **morsetto di uscita**.

EStop_Ch1 AT %IO.0: «prendi lo stato del segnale fisico che arriva all'ingresso numero 0.0 del modulo di ingresso 0 e rendilo disponibile nella variabile EStop_Ch1».

SafeOut := TRUE attiva fisicamente l'uscita (%Q0.0) che comanda il contattore (alimenta il motore); SafeOut := FALSE la disattiva (interrompe l'alimentazione al motore).

12.2.4 Analisi del comportamento

Quando il pulsante di emergenza viene premuto, uno o entrambi i canali EStop_Ch1/Ch2 passano a FALSE (contatto aperto): la logica forza immediatamente SafeOut := FALSE disattivando il contattore (arresto del motore).

La variabile EStop_Latch garantisce che il sistema resti nello stato di arresto di emergenza finché l'operatore non resetta manualmente: anche se il pulsante viene rilasciato (i canali tornano TRUE), l'uscita di sicurezza non si riattiva finché non viene premuto il pulsante di reset. Questa misura previene **riavvii automatici indesiderati**: il riavvio dopo un'emergenza deve avvenire solo tramite un'**azione volontaria**.

12.2.5 Tabella di verità con guasti

La condizione di stop è (NOT EStop_Ch1) OR (NOT EStop_Ch2):

EStop_Ch1	EStop_Ch2	NOT Ch1	NOT Ch2	SafeOut (stop)
0	0	1	1	1 (stop)
0	1	1	0	1 (stop)
1	0	0	1	1 (stop)
1	1	0	0	0 (ok)

Basta che *un solo* canale si apra (guasto o pressione) perché la funzione di sicurezza scatti.

12.2.6 Perché i contatti NC rendono il sistema fail-safe

In caso di guasto o rottura di un componente (sensore, cablaggio) il sistema si porta automaticamente in uno stato sicuro:

- A. In condizioni normali (senza guasti) il segnale è **TRUE** (circuito chiuso, corrente che passa);
- B. Se il sensore si guasta (cavo rotto o scollegato) il circuito si apre → corrente interrotta → segnale immediatamente **FALSE**.

Poiché la logica di sicurezza interpreta lo stato **FALSE** come situazione di pericolo o guasto, ogni interruzione accidentale porta automaticamente il sistema in condizione di sicurezza. Il sensore NC usa la **presenza di corrente** (**TRUE**) come indicatore di «tutto a posto» e la **manca di corrente** (**FALSE**) come indicatore di «pericolo o guasto».

EXAMPLE I tre scenari del contatto NC

- **Normale:** pulsante non premuto → contatto NC chiuso → %I0.0 = **TRUE** → «situazione normale», motore abilitato;
- **Emergenza:** pulsante premuto → contatto NC si apre → %I0.0 = **FALSE** → sistema rileva immediatamente l'emergenza, motore fermato;
- **Guasto accidentale:** cavo rotto o connettore staccato → circuito aperto → %I0.0 = **FALSE** → interpretato immediatamente come emergenza, motore fermato.

E CON UN SENSORE NO (NORMALLY OPEN)?

Con un sensore NO (normalmente **FALSE**): normale funzionamento = contatto aperto (%I = **FALSE**); emergenza = contatto chiuso (%I = **TRUE**). Problema: se il filo si rompe, il sistema vede comunque **FALSE**, che per un contatto NO significa «tutto normale». Un guasto passerebbe **inosservato**, creando una situazione potenzialmente pericolosa. Per questo i contatti NO non sono tipicamente impiegati per sensori critici di sicurezza.

12.2.7 Gli attuatori: principio fail-safe

Per un attuatore (contattore, valvola, relè di sicurezza) il principio fail-safe significa che, in caso di guasto, perdita di alimentazione o interruzione del comando, l'attuatore assume automaticamente la **posizione più sicura possibile**:

- stato sicuro = **assenza** di alimentazione (contatto aperto o valvola chiusa);
- stato operativo normale = **presenza** di alimentazione (contatto chiuso o valvola aperta).

DEFINITION Contattore di sicurezza

Particolare tipo di relè elettromeccanico progettato specificamente per funzioni di sicurezza: quando attivato dalla funzione di sicurezza (ad esempio un arresto di emergenza) apre il circuito di potenza e **scollega fisicamente il motore** dall'alimentazione elettrica. È «di sicurezza» perché ha caratteristiche che garantiscono un'apertura affidabile e sicura del circuito (ad esempio **contatti a guida forzata** che non possono bloccarsi in posizione chiusa).

Funzionamento: quando il PLC di sicurezza invia **TRUE**, la bobina si eccita e chiude il contatto (alimentazione al motore, condizione normale); in emergenza il PLC invia **FALSE**, la bobina si diseccita e il contatto si apre immediatamente, arrestando il movimento. Se si rompe un filo o si guasta la bobina, il contattore si diseccita automaticamente (non riceve alimentazione), aprendo il circuito e mettendo il sistema in sicurezza.

DEFINITION STO — Safe Torque Off

Funzione di sicurezza inclusa in molti azionamenti elettronici (inverter): quando il segnale STO è attivato dalla funzione di sicurezza, l'inverter **interrompe immediatamente la generazione della coppia motrice**, anche se rimane alimentato elettricamente. Il motore non può produrre movimenti pericolosi senza rimuovere fisicamente la tensione dall'inverter. Soluzione molto affidabile, tipicamente certificata fino a SIL 3 o PL e.

12.2.8 Categoria di sicurezza della funzione E-Stop

I due canali ridondanti permettono di **tollerare un singolo guasto**: se un contatto del pulsante si guastasse bloccandosi chiuso, l'altro canale può ancora comandare lo stop. Inoltre, con opportune verifiche di **coerenza tra i due canali** è possibile individuare guasti latenti (diagnostica): ad esempio si può monitorare la **simultaneità dei cambi di stato** dei due ingressi. In applicazioni reali si usa il blocco funzione **SF_Equivalent** della libreria di sicurezza PLCopen per controllare che i due canali dell'E-Stop commutino entro un certo tempo, rilevando eventuali discordanze.

L'architettura implementata è a **due canali ridondanti con rilevamento di guasto** (confronto dei canali): corrisponde a una **Categoria 3 o 4** (ISO 13849-1) a seconda del grado di diagnostica — se ogni singolo guasto non provoca la perdita della funzione di arresto ed è possibilmente rivelato (Cat. 3) o sempre rivelato prima della successiva richiesta (Cat. 4). La funzione può raggiungere un PL molto alto (**PL d o PL e**): una funzione di E-Stop a due canali con componenti molto affidabili e diagnostica sufficiente può ottenere Categoria 3, PL e, come spesso richiesto per arresti di emergenza in macchine ad alto rischio.

12.3 Esempio 2: Funzione di Controllo Interblocco di un Riparo

12.3.1 Descrizione

Un **interblocco di riparo** è una funzione di sicurezza che monitora lo stato di un riparo mobile (sportello o cancello di protezione) e assicura che la macchina si fermi quando il riparo è aperto. Spesso include un dispositivo di interlock:

- la macchina non deve poter funzionare se il riparo non è chiuso e bloccato;
- il riparo rimane bloccato (chiuso) finché l'area pericolosa non è in condizioni sicure (movimenti pericolosi fermi).

La funzione protegge l'operatore impedendogli l'accesso a parti pericolose in movimento e prevenendo riaccensioni accidentali quando il riparo è aperto.

12.3.2 Sensori e attuatori

- riparo dotato di **finecorsa di sicurezza a doppio canale** (due sensori o contatti ridondanti) per rilevare lo stato chiuso;
- **attuatore di bloccaggio elettromagnetico** (elettroserratura) che tiene chiuso il riparo durante il funzionamento;
- motore controllato da un **contattore di sicurezza** (lo stesso dell'E-Stop, ma comandato dall'interlock);
- segnale di **richiesta sblocco** azionato dall'operatore (pulsante di richiesta accesso) e **pulsante di reset**.

12.3.3 Programma ST

```
PROGRAM Safety_Guard
VAR
  GuardClosed_Ch1 AT %I0.3 : BOOL; // Finecorsa riparo, canale 1 (TRUE = chiuso)
  GuardClosed_Ch2 AT %I0.4 : BOOL; // Finecorsa riparo, canale 2
  UnlockRequest AT %I0.5 : BOOL; // Pulsante richiesta sblocco riparo
  ResetGuard AT %I0.6 : BOOL; // Pulsante di reset per il riparo
  LockSolenoid AT %Q0.1 : BOOL; // Uscita elettroserratura (TRUE = bloccato)
  SafeMotorEnable AT %Q0.2 : BOOL; // Abilitazione sicura motore (contattore/STO)
  Guard_Latch : BOOL := FALSE; // Latch stato riparo aperto/fault
END_VAR

// Monitoraggio: se il riparo si apre (o un canale rileva aperto), arresta
IF (NOT GuardClosed_Ch1) OR (NOT GuardClosed_Ch2) THEN
  SafeMotorEnable := FALSE; // disabilita immediatamente il motore
  Guard_Latch := TRUE;      // stato di fault/porta aperta (richiedera' reset)
END_IF;

// Blocco del riparo durante il funzionamento
IF SafeMotorEnable THEN
```

```

    LockSolenoid := TRUE; // chiude elettroserratura (riparo bloccato)
ELSE
    LockSolenoid := FALSE; // motore non attivo: sblocca il riparo
END_IF;

// Mantiene il motore fermo finché il riparo non è chiuso e reset effettuato
IF Guard_Latch THEN
    SafeMotorEnable := FALSE;
END_IF;

// Reset: riparo di nuovo chiuso (entrambi i canali) e reset premuto
IF Guard_Latch AND GuardClosed_Ch1 AND GuardClosed_Ch2 AND ResetGuard THEN
    Guard_Latch := FALSE;
END_IF;

// Riabilitazione motore: solo riparo chiuso & bloccato e nessun latch attivo
IF (NOT Guard_Latch) AND GuardClosed_Ch1 AND GuardClosed_Ch2
    AND LockSolenoid THEN
    SafeMotorEnable := TRUE;
END_IF;

```

12.3.4 Analisi del comportamento

Quando il riparo è aperto (uno dei sensori va a FALSE) la logica porta immediatamente **SafeMotorEnable** a FALSE (arresto sicuro del motore) e memorizza lo stato di interblocco aperto in **Guard_Latch**. Simultaneamente la bobina di blocco **LockSolenoid** viene rilasciata (FALSE) per consentire l'apertura del riparo. Finché il riparo non torna chiuso e non si effettua il reset manuale, **Guard_Latch** rimane TRUE e impedisce la riattivazione del motore.

La sequenza garantisce che:

1. la macchina **si ferma sempre** se il riparo non è chiuso/bloccato;
2. il riparo può sbloccarsi **solo** quando la macchina è in stato sicuro (motore fermo);
3. la ripartenza richiede che il riparo sia richiuso e un reset sia premuto: niente riavvii automatici pericolosi.

12.3.5 Categoria di sicurezza della funzione di interblocco

Anche l'interblocco di riparo è realizzato con due canali di rilevamento e blocco, analogamente all'E-Stop, e può raggiungere **Categoria 4, PL** e impiegando componenti ad alta affidabilità e diagnostica completa. Nei ripari interbloccati si tende a richiedere il massimo livello (PL e), dato il rischio di accesso dell'operatore a parti pericolose: tipicamente si usano sensori ridondanti e autodiagnosticati, e si implementano misure contro i guasti da causa comune (CCF) per soddisfare la Categoria 4.

12.3.6 Integrazione di sensori e attuatori di sicurezza

- I sensori di sicurezza sono spesso **NC** per essere fail-safe (un'interruzione del circuito causa immediatamente FALSE, rilevato come guasto). Nel codice bisogna tenere conto della logica inversa dei NC (come fatto con **NOT EStop_ChX**);
- gli attuatori di sicurezza spesso richiedono una **logica attiva a livello basso** per il fail-safe (es. un contattore si eccita con TRUE per dare energia: FALSE = sicuro).

12.4 PLC di sicurezza (Safety PLC)

DEFINITION Safety PLC

Controllori logici programmabili specificamente **progettati, realizzati e certificati** per realizzare funzioni di sicurezza in applicazioni industriali dove occorre garantire affidabilità assoluta, evitando che un guasto o un errore possa portare a situazioni pericolose per persone, macchinari o ambiente. Rispetto ai normali PLC hanno caratteristiche specifiche, certificate e garantite secondo standard.

Caratteristiche principali:

- **Architettura ridondante interna:** spesso almeno due processori interni lavorano in parallelo, elaborando le stesse informazioni e confrontandole continuamente per identificare eventuali guasti hardware, portando il sistema verso uno stato fail-safe (nota: soltanto quando esiste effettivamente uno stato fail-safe fisicamente raggiungibile!);
- **Autodiagnostica integrata:** test automatici continui su memoria, processori, ingressi e uscite; rilevano immediatamente guasti o anomalie e reagiscono portando il sistema in stato sicuro;
- **Certificazione di sicurezza:** progettati, testati e certificati (ad esempio dal TÜV) per soddisfare livelli precisi di sicurezza (SIL 2, SIL 3, PL d, PL e), con documentazione certificata del produttore;
- **Ingressi e uscite di sicurezza:** moduli dedicati, ridondanti, con diagnostica integrata;
- **Librerie software certificate:** funzioni certificate per le principali funzioni di sicurezza (arresto emergenza, monitoraggio ripari, interblocchi, comandi a due mani, ecc.).

IL SOFTWARE DI UN PLC DI SICUREZZA È SICURO ANCH'ESSO?

Il software di un PLC di sicurezza può rappresentare il **punto debole** (e quindi pericoloso) dell'intera architettura dedicata alla sicurezza. Il linguaggio ST viene usato solo per separare «zone» di codice con diverse funzionalità durante l'esecuzione: si passa attraverso diverse fasi (attuazione, gestione, uscita dall'emergenza) con un linguaggio che prevede costrutti sintattici al pari di C o Pascal, ma di cui viene utilizzato **solo il costrutto IF** per simulare un formalismo a stati. Può non essere semplice distinguere le condizioni di attivazione e di permanenza delle diverse fasi. Un **linguaggio basato sugli stati**, poi tradotto in ST attraverso una struttura **CASE OF**, rende la progettazione del software più semplice e sicura.

Capitolo 13

Robotica collaborativa

13.1 Cobot: definizione e motivazione

DEFINITION Cobot — Collaborative Robot

A differenza dei robot industriali tradizionali, che operano **separati** dagli esseri umani per motivi di sicurezza (recinzioni di sicurezza, *safety fences*), i cobot lavorano **a fianco degli operatori**. Sono progettati per adattarsi in tempo reale alle azioni umane, rendendo la produzione più flessibile ed efficiente.

«Aprire le recinzioni» è parte integrante della natura collaborativa del nuovo paradigma robotico: i robot collaborativi richiedono in maniera determinante di porre l'enfasi sulla sicurezza e richiedono standard specifici (ISO 10218-1:2011 e specifica tecnica ISO/TS 15066).

13.2 Origine dei cobot: problemi ergonomici

Negli anni '90, l'**OSHA** (Occupational Safety and Health Administration) era preoccupata per il modo in cui GM e l'industria manifatturiera gestivano i problemi ergonomici negli stabilimenti:

- i problemi ergonomici si trasformavano in infortuni sul lavoro e perdita di tempo di lavoro per i dipendenti;
- i problemi colpivano le case automobilistiche in tutti gli Stati Uniti, ma per GM erano più significativi nelle aree di assemblaggio finale;
- esempio: prendere e installare una batteria per auto da 20 kg, una al minuto per otto ore al giorno, 200 giorni all'anno, mette a dura prova la spina dorsale.

La prima risposta fu lo **Z-Lift Assist** (dispositivo di assistenza al sollevamento verticale). Da qui nacquero le **macchine a vincoli programmabili**:

- obiettivo: migliorare l'ergonomia dei lavoratori umani **senza introdurre nuovi rischi** da parte dei robot stessi;
- robot e persone potevano lavorare in collaborazione, contribuendo ciascuno con ciò che sa fare meglio;
- esempio: il robot sostiene un carico ma non lo muove, e la forza motrice viene dal lavoratore umano;
- questi primi cobot erano chiamati «macchine a vincoli programmabili» perché il robot sosteneva il peso di un oggetto lungo assi o superfici determinate, quindi vincolate.

13.3 Spazio di lavoro collaborativo (ISO 15066)

Le caratteristiche operative dei sistemi robotici collaborativi sono significativamente diverse da quelle dei sistemi robotici tradizionali: nelle operazioni collaborative gli operatori possono lavorare in prossimità del sistema robotico **mentre è disponibile l'alimentazione agli attuatori**, e il contatto fisico tra operatore e sistema robotico può avvenire all'interno dello **spazio di lavoro collaborativo**.

DEFINITION Spazio di lavoro collaborativo

È lo spazio all'interno dello spazio operativo in cui il sistema robotico e un essere umano possono **eseguire attività contemporaneamente**.

13.4 Forme differenti di co-lavoro

Le forme di co-lavoro si distinguono per tre caratteristiche: spazio separato o condiviso, co-work simultaneo, contatto fisico:

Forma	Spazio condiviso	Co-work simultaneo	Contatto fisico
Coexistence	No	No	No
Sequential Cooperation	Sì	No	No
Parallel Cooperation	Sì	Sì	No
Collaboration	Sì	Sì	Sì

13.4.1 Coexistence (spazio separato)

Il lavoratore umano svolge un certo compito in una parte **separata, non sovrapposta** dello spazio di lavoro del robot. Caso banale.

13.4.2 Shared Workspace e Sequential Cooperation

Shared workspace: il co-worker umano svolge un certo compito in almeno una parte limitata dello spazio di lavoro del robot. Descrive solo una disposizione **spaziale** dei due agenti, senza condizioni temporali (nessuna coordinazione mutua).

Sequential Cooperation: stato o condizione di sforzi condivisi e azione congiunta **successiva**; nel manifatturiero si riferisce ad azioni sequenziali per completare un compito comune. Esempio: robot e umano lavorano successivamente sullo stesso pezzo. Mentre l'umano lavora il robot è fermo, e mentre il robot lavora l'umano non è nello spazio condiviso.

13.4.3 Simultaneous Co-Work e Parallel Cooperation

Simultaneous co-work: l'umano svolge un compito nello spazio condiviso **mentre il robot si muove** per completare il suo. Robot e umano lavorano separatamente: il contatto fisico non è richiesto e va evitato. Lo spazio in cui il robot si sta muovendo deve essere protetto (*guarded*) contro la violazione da parte del co-worker umano; il contatto fisico è considerato un **incidente**.

Parallel Cooperation: sforzi condivisi e azione **simultanea** per completare un compito comune; robot e umano lavorano simultaneamente sullo stesso pezzo nello spazio condiviso, senza contatto fisico previsto o permesso (esplicitamente escluso).

13.4.4 Collaboration (contatto fisico)

Il co-worker umano lavora **mano nella mano** con il robot in movimento; il contatto fisico tra i due agenti è possibile e persino **necessario** per completare il compito comune. Entrambi gli agenti devono necessariamente condividere lo spazio e il co-work simultaneo. La **collaboration** è la forma più stretta di cooperazione: azioni congiunte simultanee per completare un compito comune.

13.5 Operazioni collaborative: i quattro metodi

Le operazioni collaborative possono includere uno o più dei seguenti metodi:

1. Hand Guiding;
2. Safety Monitored Stop;
3. Speed and Separation Monitoring;
4. Power and Force Limiting.

13.5.1 Hand Guiding

Caratteristica collaborativa usata per **insegnare (teaching)** al robot: guidare letteralmente il robot attraverso una sequenza di movimenti necessari a completare un compito, come applicazioni pick-and-place. I cobot usano spesso tecnologia nell'end effector per percepire la propria posizione e leggere le forze applicate agli utensili, permettendo al cobot di **imparare dall'esempio**. (Esempio classico: hand guiding del robot KUKA.)

13.5.2 Safety Monitored Stop

Usata in cobot progettati per lavorare **prevalentemente da soli**, con l'umano che entra raramente nel loro spazio di lavoro. Esempio: se un dipendente deve regolare un pezzo gestito dal cobot o rimuoverlo dal suo spazio, il cobot **cessa il movimento ma non si spegne completamente**, per riprendere facilmente i compiti quando l'umano se ne va. È l'interpretazione più «lasca» di robot collaborativo: impiega tecnologia avanzata di sensoristica di prossimità per trasformare un robot industriale convenzionale in una forma di cobot.

13.5.3 Speed and Separation Monitoring

Per applicazioni che richiedono interventi umani più frequenti con cobot più grandi: un **sistema di visione avanzato** installato nell'ambiente permette al robot di percepire la prossimità umana. Il robot **rallenta progressivamente** man mano che un umano si avvicina, fermandosi completamente quando gli umani sono troppo vicini. Queste reazioni possono essere programmate a varie distanze tra robot e operatore. Il robot riprende il compito e ri-accelera lentamente man mano che l'operatore si allontana.

SAFETY-RATED SPACE LIMITING

Limite posto sulla distanza tra l'umano e il raggio di movimento del robot da un sistema **software- o firmware-based** (certificato per la sicurezza): è il meccanismo alla base di Safety Monitored Stop e Speed and Separation Monitoring.

13.5.4 Power and Force Limiting

La tecnologia di sicurezza è spesso quasi completamente basata su funzioni di limitazione di potenza e forza:

- i robot con forza limitata leggono le forze nei propri giunti (pressione, resistenza, impatti) usando **sensori integrati**; dopo aver percepito una perturbazione, il robot si **ferma o inverte** il proprio moto;
- la **pelle morbida** (meccanismo *passivo*) e i **tempi di reazione molto rapidi** (*attivo*) dissipano quanto più possibile l'impatto;
- pelli morbide e design più arrotondati aiutano ad ammortizzare gli impatti e persino a eliminare i punti di schiacciamento (*pinch points*).

DEFINITION Contatti transienti vs. quasi-statici

1. Un impatto **quasi-statico** si verifica quando il robot schiaccia una parte del corpo umano *contro un oggetto fisso*;
2. un impatto **transiente** si verifica quando il robot entra in contatto con una parte del corpo umano *senza alcuna restrizione*: la parte del corpo può muoversi liberamente sotto la forza del robot.

Regola generale: un impatto transiente può applicare una forza **due volte superiore** a quella di un impatto quasi-statico. Esempio: se il robot schiaccia la mano contro un tavolo, la forza massima prima di raggiungere la soglia del dolore umano è 135 N; nell'impatto transiente si può arrivare a 270 N prima della soglia del dolore.

13.6 Misure protettive in robotica collaborativa

Se la distanza di separazione tra una parte pericolosa del sistema robotico e un qualsiasi operatore scende sotto la **distanza di separazione protettiva** (*protective separation distance*), il sistema robotico deve:

- iniziare un **arresto protettivo** (safety-monitored stop);
- attivare le funzioni di sicurezza collegate al sistema robotico (es. spegnere eventuali utensili pericolosi).

Le possibilità con cui il sistema di controllo del robot può evitare di violare la distanza di separazione protettiva includono (ma non si limitano a):

- **riduzione della velocità**, eventualmente seguita da una transizione a un safety-rated monitored stop;
- esecuzione di un **percorso alternativo** che non violi la distanza di separazione protettiva, continuando con speed and separation monitoring attivo.

Quando la distanza di separazione effettiva raggiunge o supera la distanza di separazione protettiva, il moto del robot può essere ripreso.

13.6.1 Mantenimento della distanza di separazione sufficiente

Durante il funzionamento automatico, le parti pericolose del sistema robotico non devono mai avvicinarsi all'operatore più della distanza di separazione protettiva. Questa può essere calcolata tenendo conto dei seguenti pericoli associati allo speed and separation monitoring:

- in situazioni di velocità **costante**: si usa il valore *worst-case* della velocità monitorata del robot (dipende dall'applicazione);
- in situazioni di velocità **variabile**: si usano le velocità del sistema robotico e dell'operatore per determinare il valore applicabile della distanza di separazione protettiva a ogni istante.

13.6.2 Calcolo della distanza protettiva S_p

La S_p può essere calcolata in modo **dinamico** (consentendo alla velocità del robot di variare durante l'applicazione) o **statico** (valore fisso basato su valori worst-case):

$$S_p = S_h + S_r + S_s + C + Z_d + Z_r$$

dove:

- S_h : variazione di posizione dell'**operatore** (*operator's change in location*);
- S_r : spazio dovuto al **tempo di reazione** del sistema robotico;
- S_s : **distanza di arresto** (*stopping distance*) del sistema robotico;
- C : **distanza di intrusione**: la distanza che una parte del corpo può penetrare nel campo di rilevamento prima di essere rilevata;
- Z_d : **incertezza di posizione dell'operatore** nello spazio collaborativo, così come misurata dal dispositivo di rilevamento presenza (tolleranza di misura del sistema di sensing);
- Z_r : **incertezza di posizione del sistema robotico**, risultante dall'accuratezza del sistema di misura della posizione del robot.

Dettagli sui primi due termini:

- S_h rappresenta il contributo dovuto al moto dell'operatore dal tempo corrente fino all'arresto del robot; è calcolato secondo la velocità relativa v_h , funzione del tempo, che può variare per cambio di velocità o direzione della persona. Se la velocità della persona **non è monitorata**, il progetto deve assumere $v_h = 1,6$ m/s nella direzione che riduce maggiormente la distanza di separazione;
- S_r rappresenta il contributo dovuto al moto del robot dall'ingresso della persona nel campo di rilevamento fino all'attivazione dello stop da parte del sistema di controllo; è calcolato secondo la velocità relativa v_r , funzione del tempo; il sistema deve essere progettato per tenere conto di v_r variabile nel modo che riduce maggiormente la distanza di separazione.

RIFERIMENTI

ISO STANDARD 10218, *Robots and robotic devices — Safety requirements for industrial robots*; ISO/TS 15066, *Robots and robotic devices — Collaborative robots*; Björn Matthias (ABB Corporate Research), *Collaborative Robots — Power and Force Limiting*; Jeff Fryman, Björn Matthias, *Safety of Industrial Robots: From Conventional to Collaborative Applications*, ROBOTIK 2012, Munich; engineering.com, *A History of Collaborative Robots: From Intelligent Lift Assists to Cobots*; documentazione Omron sul collaborative robot rated stop.